ELDERGUARD AI

Connected Elder Care Platform: Medicine Reminders, Tiered SOS Escalation and Five Linked Care Portals

A MINI PROJECT REPORT

Submitted by

DINESH N	**713525AD029**
BHOOPESH M	**713525AD013**
DHARSHAN V	**713525AD014**

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

SNS COLLEGE OF TECHNOLOGY

COIMBATORE 641035

October 2026

SNS COLLEGE OF TECHNOLOGY

COIMBATORE 641035

BONAFIDE CERTIFICATE

Certified that this mini Project Report titled, "ELDERGUARD AI – Connected Elder Care Platform" is the bonafide record of **DINESH N, BHOOPESH M and DHARSHAN V** who carried out the mini Project Work under our supervision. Certified further, that to the best of my knowledge the work reported herein does not form part of any other mini project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROJECT GUIDE

Ms. K. Revathi
Assistant Professor,
Department of AI & DS,
SNS College of Technology
Coimbatore-641035.

HEAD OF THE DEPARTMENT

Dr. A. Kowshika
Associate Professor & Head,
Department of CSE,
SNS College of Technology
Coimbatore-641035.

Submitted for the Viva-Voce examination held at SNS COLLEGE OF TECHNOLOGY, held on .....

Examiner 1 Examiner 2

ABSTRACT

Many older adults live apart from their families. Missed medicines, unnoticed changes in blood pressure or blood sugar, and slow responses to falls and sudden illness put their health and independence at risk. At the same time the people who look after them, namely family caregivers, doctors, pharmacies and hospitals, each work from separate and disconnected information, so prescriptions, vitals and emergency alerts rarely reach the right person in time.

ElderGuard AI is a connected elder care platform that places all of these people on one shared, live record. It is a web application with five role-based portals: **Elder**, **Caregiver**, **Doctor**, **Hospital** and **Pharmacy**. The elder portal has extra-large type and a one-tap SOS button. It rings a medicine reminder until the dose is answered, lets the elder log vitals, and offers a friendly AI companion named Mevi. The caregiver portal shows the elder's medicine adherence, vitals trends and alerts in real time. Doctors manage patients, telemedicine appointments with queue tokens, and e-prescriptions. A prescription written for a named pharmacy appears in that pharmacy's order queue, where it can be verified and fulfilled. The hospital portal handles the appointment queue, doctors, departments, billing and live SOS dispatch.

The central safety feature is a tiered SOS escalation. When the elder presses SOS, the caregiver is alerted immediately. If nobody acknowledges the alert, a scheduler escalates to a second contact after 2 minutes, to the hospital after 5 minutes and to emergency services after 10 minutes. A demo mode compresses these timings so the whole chain can be shown in under a minute.

The system is built with a React 18, Vite and Tailwind CSS front end and a FastAPI back end. Data is held in SQLite for local use and in Supabase/PostgreSQL when it is configured. WebSockets deliver live updates, APScheduler runs the reminder and escalation jobs, and a Groq-hosted language model powers the companion. The interface follows a single warm design language with large type and touch targets, and is checked against WCAG 2.1 A/AA rules. The prototype is verified by 22 automated Playwright end-to-end tests across all five portals, an automated axe accessibility audit with no serious violations, and a timed drill in which all four SOS tiers fired in order.

ElderGuard alerts and informs; it does not diagnose and does not replace a doctor or emergency services. This report documents the problem, user research approach, ideation, requirements, design, implementation, testing and evaluation of the prototype, and sets out the work needed to take it to a real pilot.

Keywords: Elder Care, Medicine Reminder, SOS Escalation, Connected Care, Accessibility, FastAPI, React, Telemedicine, e-Prescription.

TABLE OF CONTENTS

1. Introduction..... 1

1.1 Background..... 1

1.2 Project Overview..... 1

1.3 Motivation..... 2

1.4 Objectives..... 2

1.5 Scope..... 3

2. Problem Identification..... 4

2.1 Existing Scenario..... 4

2.2 Existing System..... 4

2.3 Existing System Drawbacks..... 5

2.4 Problem Statement..... 5

2.5 Proposed Solution..... 5

3. Empathize and Define..... 6

3.1 Empathy Study..... 6

3.2 User Identification..... 6

3.3 Primary Users..... 6

3.4 Secondary Users..... 6

3.5 User Needs..... 6

3.6 Pain Points..... 7

3.7 User Persona..... 7

3.8 User Expectations..... 7

3.9 Refined Problem Definition..... 8

4. Ideation..... 9

4.1 Idea Generation..... 9

4.2 Brainstorming..... 9

4.3 Evaluation of Ideas..... 9

4.4 Selected Idea.....10
4.5 Key Features Identified.....10
4.6 Final Concept.....10
5. Requirements Analysis.....11
5.1 Introduction.....11
5.2 Functional Requirements.....11
5.3 Non-Functional Requirements.....11
5.4 Hardware Requirements.....12
5.5 Software Requirements.....12
5.6 User Roles and Permissions.....12
5.7 System Constraints.....13
5.8 Requirement Summary.....13
6. Technology Stack.....14
6.1 Introduction.....14
6.2 Frontend Technologies.....14
6.3 Backend Technologies.....15
6.4 Database Technology.....15
6.5 Artificial Intelligence.....15
6.6 Notification Technology.....15
6.7 Technology Integration.....16
6.8 Technology Selection.....16
7. System Design.....17
7.1 System Architecture.....17
7.2 Introduction.....17
7.3 Working Flow.....17
7.4 System Components.....18
7.5 Use Case Diagram.....18

7.6 SOS Escalation Flow.....19
7.7 Prescription Handoff Flow.....19
7.8 Data Flow.....20
7.9 System Design Considerations.....20
8. Database Design.....21
8.1 Introduction.....21
8.2 Database Objectives.....21
8.3 Main Database Entities.....21
8.4 User Table.....22
8.5 Elder Profile Table.....22
8.6 Prescription Table.....23
8.7 SOS Event Table.....23
8.8 Appointments, Medicines and Vitals.....23
8.9 Database Relationships.....23
8.10 Database Security.....24
8.11 Database Summary.....24
9. Module Description.....25
9.1 Introduction.....25
9.2 Authentication Module.....25
9.3 Elder Portal.....25
9.4 Medicine Reminder Module.....26
9.5 Vitals Module.....26
9.6 SOS Escalation Module.....26
9.7 Caregiver Portal.....26
9.8 Doctor Portal.....27
9.9 Pharmacy Portal.....27
9.10 Hospital Portal.....27

9.11 AI Companion Module.....27
9.12 Module Integration.....28
10. Implementation and Working Principle.....29
10.1 Introduction.....29
10.2 Frontend Implementation.....29
10.3 Backend Implementation.....29
10.4 Reminder Processing.....30
10.5 SOS Processing.....30
10.6 Prescription Handoff.....30
10.7 Demo Data and Demo Mode.....31
10.8 AI Companion.....31
10.9 Implementation Summary.....32
11. Testing.....33
11.1 Introduction.....33
11.2 Objectives of Testing.....33
11.3 Types of Testing.....33
11.4 Functional Test Cases.....34
11.5 Defects Found and Fixed.....34
11.6 Testing Result.....35
11.7 Conclusion of Testing.....35
12. Project Evaluation.....36
12.1 Introduction.....36
12.2 Objective Evaluation.....36
12.3 Usability Evaluation.....36
12.4 Performance and Reliability Evaluation.....36
12.5 Security Evaluation.....37
12.6 Advantages.....37

12.7 Limitations....37
12.8 Overall Evaluation....37
13. Conclusion and Future Enhancements....38
13.1 Conclusion....38
13.2 Future Enhancements....38
Appendix A – Results and Screenshots....39
A.1 Introduction....39
A.2 Landing and Sign-in....39
A.3 Elder Portal....40
A.4 Caregiver Portal....42
A.5 Doctor Portal....43
A.6 Hospital Portal....44
A.7 Pharmacy Portal....45
Appendix B – Core Functionality Code....47
B.1 Introduction....47
B.2 Password hashing (backend/app/core/security.py)....47
B.3 SOS escalation sweep schedule (backend/app/emergency/scheduler.py)....47
B.4 Medicine reminder sweeps (backend/app/medicines/scheduler.py)....47
B.5 Demo-speed SOS....47
B.6 Code Summary....47
Project Link....48
References....49

LIST OF TABLES

Table 2.1: Comparison of Existing Approaches and ElderGuard AI.....4

Table 3.1: User Pain Points and Proposed Solutions.....7

Table 3.2: Illustrative User Personas.....7

Table 4.1: Evaluation of Candidate Ideas.....9

Table 5.1: Functional Requirements of ElderGuard AI.....11

Table 5.2: Non-Functional Requirements of ElderGuard AI.....11

Table 5.3: User Roles and Permissions.....12

Table 6.1: Technology Stack Used in ElderGuard AI.....14

Table 8.1: Users Table.....22

Table 8.2: Elder Profiles Table.....22

Table 8.3: Doctor Prescriptions Table.....23

Table 8.4: SOS Events Table.....23

Table 9.1: ElderGuard Module Description.....25

Table 11.1: Functional Test Cases.....34

Table 11.2: Notable Defects Found During Testing.....34

Table 12.1: Project Objective Evaluation.....36

Table 13.1: Roadmap.....38

LIST OF FIGURES

Figure 1.1: ElderGuard landing page....2

Figure 7.1: System architecture of ElderGuard AI....17

Figure 7.2: Use cases by role....19

Figure 7.3: SOS escalation workflow....19

Figure 7.4: Doctor-to-pharmacy prescription handoff....20

Figure 7.5: Data flow diagram of ElderGuard AI....20

Figure 8.1: Entity relationship diagram of ElderGuard AI....22

Figure 9.1: Elder dashboard....26

Figure 9.2: Caregiver dashboard....27

Figure 10.1: Role-based sign-in screen....29

Figure 10.2: Pharmacy queue showing prescriptions received from the doctor....31

Figure 10.3: AI companion screen....32

Figure A.1: Landing page....39

Figure A.2: Sign-in screen....40

Figure A.3: Elder dashboard....40

Figure A.4: Elder medicines....41

Figure A.5: Elder vitals....41

Figure A.6: Caregiver overview....42

Figure A.7: Vitals and health trends....42

Figure A.8: Doctor dashboard....43

Figure A.9: Connected patient workspace....43

Figure A.10: Appointments and telehealth desk....44

Figure A.11: Hospital operational command....44

Figure A.12: Emergency and SOS dispatch....45

Figure A.13: Pharmacy operations command....45

Figure A.14: Prescription orders and fulfilment....46

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

AI	Artificial Intelligence
API	Application Programming Interface
APScheduler	Advanced Python Scheduler
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram
ER	Entity Relationship
FCM	Firebase Cloud Messaging
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MVP	Minimum Viable Product
OCR	Optical Character Recognition
REST	Representational State Transfer
SOS	Emergency distress signal
SQL	Structured Query Language
UI	User Interface
UX	User Experience
WCAG	Web Content Accessibility Guidelines
WS	WebSocket

Chapter 1

1. Introduction

1.1 Background

Population ageing is one of the quiet changes of our time. In India, joint families are giving way to nuclear households, and children often study or work in other cities while a parent stays at home. An older adult may take three or more medicines a day for blood pressure, diabetes or cholesterol, and any of them can be forgotten. A reading that drifts upward for several days may go unnoticed. A fall at night may be discovered hours later.

The people who could help are not short of goodwill, but they are short of shared information. A daughter in another city does not know whether the morning tablet was taken. The doctor does not see the blood pressure log. The pharmacy receives a paper prescription that may be unclear. The hospital learns of an emergency only when someone phones. Each party holds one piece of the picture and no one holds all of it.

1.2 Project Overview

ElderGuard AI is a connected elder care platform that gives every person in an elder's care circle the same live record. It is delivered as a responsive web application with five role-based portals that share one back end and one database:

- **Elder portal**: extra-large type, a medicine reminder that keeps ringing until it is answered, a one-tap SOS button, vitals logging, appointments, medical reports and a friendly AI companion called Mevi.
- **Caregiver portal**: live medicine adherence, vitals trends, alerts and SOS history, a care-team message thread and management of linked elders.
- **Doctor portal**: patient roster and workspace, telemedicine appointments with queue tokens, e-prescriptions and report upload.
- **Hospital portal**: appointment queue, doctor management, departments, billing records and an emergency SOS dispatch desk.
- **Pharmacy portal**: incoming e-prescription intake, order fulfilment, inventory, delivery tracking and patient subscriptions.

Figure 1.1 shows the public landing page that introduces the platform and sends each user to the right portal.

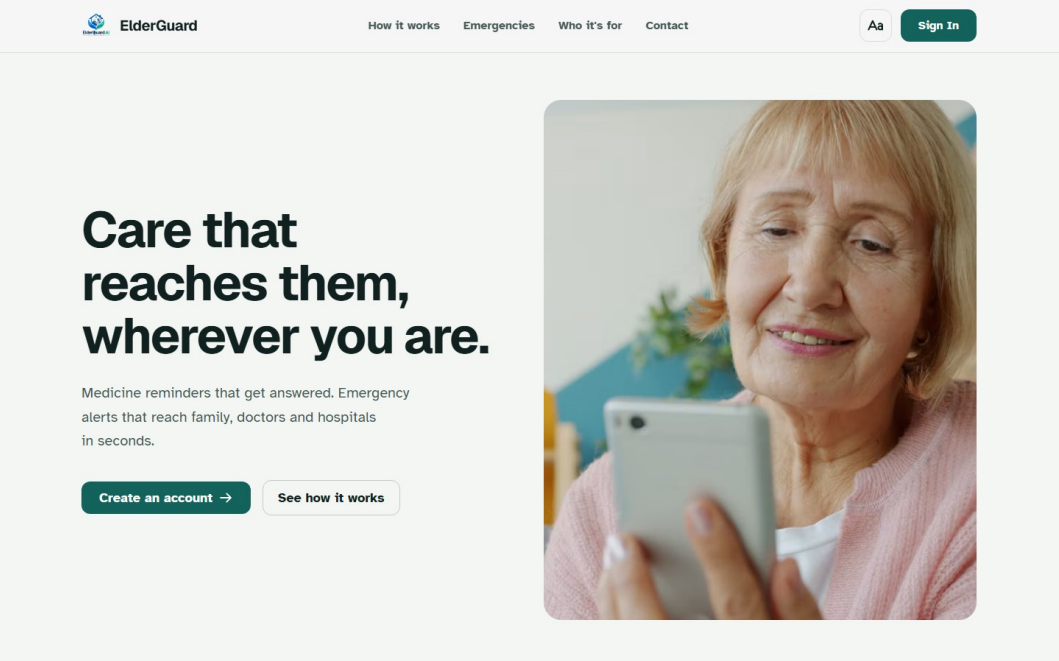


Figure 1.1: ElderGuard landing page

1.3 Motivation

The motivation for this project is the gap between care that is given and care that is seen. Families already care a great deal; what they lack is a calm, dependable way to know that medicines are being taken, that readings are normal, and that help will arrive quickly if something goes wrong. Existing alternatives either do one thing (a pill reminder, a panic pendant) or are written for clinicians rather than for the elder at home. We wanted one connected system that is simple enough for an 80-year-old to use and rich enough for a doctor and a pharmacy to rely on.

1.4 Objectives

The main objectives of ElderGuard AI are:

- To remind elders about each dose, keep reminding until it is answered, and show the caregiver the result live.
- To let an elder raise an emergency with one tap and to escalate automatically, caregiver first and then a second contact, the hospital and emergency services, if nobody responds.
- To record vitals such as blood pressure, blood sugar, oxygen level and heart rate and to flag unusual readings.

- To connect elder, caregiver, doctor, pharmacy and hospital through one shared record so that a prescription written by a doctor reaches the chosen pharmacy.
- To offer a patient AI companion that explains medicines and routines in plain language without giving a diagnosis.
- To design for older users: large type, high contrast, large touch targets and a text-size control, verified against WCAG 2.1 A/AA.
- To give each role only the information it needs through role-based access.

1.5 Scope

The scope of this mini project covers the design and implementation of a working prototype: registration and role-based sign-in, the five portals described above, medicine reminders and adherence logging, vitals, appointments with telemedicine queue tokens, the doctor-to-pharmacy prescription handoff, tiered SOS escalation with a demo-speed mode, an AI companion, real-time updates and automated tests. Items that need outside providers or real-world partners, namely SMS and voice-call delivery, wearable devices, health-record exchange and a pilot with real elders, are outside the scope and are listed under future enhancements in Chapter 13.

Chapter 2

2. Problem Identification

2.1 Existing Scenario

Today an older adult's care is spread over phone calls, paper prescriptions, notebooks, pill boxes and separate apps. A family member calls to ask whether tablets were taken and relies on the answer. A doctor sees the patient for a few minutes every few weeks and depends on memory for what happened in between. The pharmacy fills a handwritten slip. In an emergency the elder must find a phone and call someone, which is exactly what is hardest when unwell or frightened.

2.2 Existing System

Existing tools each solve part of the problem. Alarm and reminder apps ring at a set time but do not tell anyone whether the dose was taken. Panic pendants and call buttons raise an alert but do not connect to the elder's medicines, vitals or doctor. Hospital information systems are built for staff and are not designed for elders or families. Messaging groups are informal, easy to miss and keep no record. Table 2.1 compares the usual approaches with ElderGuard.

Table 2.1: Comparison of Existing Approaches and ElderGuard AI

Aspect	Existing approach	ElderGuard AI
Medicine reminder	Phone alarm; nobody knows if the dose was taken	Reminder rings until answered; caregiver sees the dose logged live
Emergency alert	Phone call or a single contact	One tap; tiered escalation to caregiver, second contact, hospital and emergency services
Vitals	Paper notebook or memory	Logged in the app; shared with caregiver and doctor; unusual readings flagged
Prescriptions	Handwritten slip carried to a shop	Doctor e-prescription routed to the named pharmacy's queue
Information sharing	Phone calls and chat groups	One shared record with role-based access
Usability for elders	Small text, many steps	Extra-large type, one clear action per screen, text-size control

2.3 Existing System Drawbacks

- A missed dose is discovered late, if at all.
- Emergency response depends on the elder being able to phone someone, and on that person answering.
- Health information is scattered across people and paper, so no one sees a trend.
- Prescriptions travel as paper and are easy to lose or misread.
- Tools for professionals are too complex for elders, and consumer apps are too shallow for clinicians.
- There is no audit trail of who was alerted and when.

2.4 Problem Statement

"Elders and their families need a simple, connected way to take medicines on time, share health information and get help fast, so that older adults can live independently with confidence."

2.5 Proposed Solution

ElderGuard AI proposes one platform with five linked portals on a shared record. The solution works in five steps: **capture** (elders log medicines and vitals, doctors write prescriptions), **connect** (one record links elder, caregiver, doctor, pharmacy and hospital), **alert** (reminders ring, unusual readings are flagged, SOS goes out in one tap), **escalate** (caregiver now, a second contact at 2 minutes, the hospital at 5 minutes and emergency services at 10 minutes) and **review** (history lets family and doctors act on what happened).

Chapter 3

3. Empathize and Define

3.1 Empathy Study

The design started from the people rather than the technology. The empathy approach for this project combines interviews with elders and family caregivers, observation of medicine routines at home, medicine-routine diary studies and journey mapping of an emergency. For this mini project the team built the findings below from structured desk research and the team's own family experience, and used them to draw an illustrative persona. **Interviews with real elders and caregivers are planned for the pilot phase and have not yet been carried out.**

3.2 User Identification

Five groups use the system, each with a different goal. The elder and the caregiver are the primary users; the doctor, hospital and pharmacy are the secondary users who turn information into treatment.

3.3 Primary Users

- **Elders** aged about 60 and above who take regular medicines and may live alone or apart from family.
- **Family caregivers**, usually adult children, who worry about missed doses and emergencies and want reassurance that does not feel like surveillance.

3.4 Secondary Users

- **Doctors** who prescribe, review vitals and run telemedicine visits.
- **Pharmacies** who receive prescriptions and fulfil orders and deliveries.
- **Hospitals** who manage appointments, doctors and the emergency dispatch desk.

3.5 User Needs

- A reminder that is hard to miss and easy to answer.
- Help in one tap, without having to find and dial a number.
- Knowing that someone will respond even if the first person does not.

- A clear view of how the elder is doing, without constant phone calls.
- A prescription that reaches the pharmacy without paper.
- Large, readable screens and plain language.

3.6 Pain Points

Table 3.1: User Pain Points and Proposed Solutions

S.No	Pain point	Proposed solution in ElderGuard
1	Several daily medicines are easy to forget	Reminder alarm that rings until answered, with adherence log
2	Family finds out about a missed dose late	Live dose status on the caregiver dashboard
3	A fall or sudden illness goes unnoticed	One-tap SOS with timed escalation
4	Blood pressure or sugar not logged for days	Easy vitals entry (typed or spoken) and trend charts
5	Prescription on paper is lost or misread	Doctor e-prescription routed to the named pharmacy
6	Doctor, pharmacy and hospital hold separate data	One shared record with role-based portals
7	Technology is hard for older users	Extra-large type, 44 px touch targets and a text-size control

3.7 User Persona

The persona below is illustrative, created from the team's research to guide design decisions. It is not a record of a real interview.

Table 3.2: Illustrative User Personas

Attribute	Mrs. Anaya Rao (Elder)	Maya (Caregiver)
Age and situation	77, lives alone, takes three medicines a day	Daughter, works in another city
Goal	Stay independent at home and not be a burden	Know that her mother is safe without calling every hour
Challenge	Forgets doses, small phone text is hard to read	Worries about night-time emergencies
Expectation	"Tell me simply what to do."	"Let me know if something is wrong."

3.8 User Expectations

- The app should feel calm and friendly, never alarming unless there is a real alert.
- Anything important must be possible with one clear tap.

- The family should be told about problems, not flooded with noise.
- Doctors and pharmacies expect the information to be accurate and traceable.

3.9 Refined Problem Definition

How might we help an older adult take medicines on time, share health information with the people who care for them, and get help fast in an emergency, using an interface that the elder can operate unaided and the family can trust?

Chapter 4

4. Ideation

4.1 Idea Generation

The team generated ideas by listing every moment in an elder's day that could go wrong (waking, meals, medicine times, nights, appointments, emergencies) and asking what information or action would help at each one.

4.2 Brainstorming

Ideas were gathered without judgement first, then grouped. The main candidates were: reminders that must be answered; a one-tap SOS; vitals tracking with an AI companion; a prescription scanner that turns a photo into a schedule; doctor appointments with queue tokens; doctor-to-pharmacy e-prescriptions; and role-based portals with a large-text mode.

4.3 Evaluation of Ideas

Table 4.1: Evaluation of Candidate Ideas

Idea	Value to elder	Feasibility in a mini project	Decision
Answerable medicine reminder	High	High	Selected
One-tap SOS with escalation	Very high	High	Selected (core feature)
Vitals tracking and trends	High	High	Selected
AI companion (Mevi)	Medium	High (hosted LLM)	Selected
Doctor-to-pharmacy e-prescription	High	Medium	Selected
Telemedicine appointments with tokens	Medium	Medium	Selected
Prescription photo scanner	Medium	Low (needs OCR service)	Prototype screen only
Wearable and fall detection	High	Low (hardware)	Future work

4.4 Selected Idea

The selected idea is a **connected care platform**: one shared record, five portals, with answerable reminders and tiered SOS escalation at its core. The decision was driven by the observation that every pain point is a symptom of disconnected information.

4.5 Key Features Identified

- Medicine reminder alarm with Taken action and caregiver visibility.
- One-tap SOS with four escalation tiers and a demo-speed mode.
- Vitals logging, target ranges and trend charts.
- Doctor portal with patient workspace, telemedicine tokens and e-prescriptions.
- Pharmacy intake of prescriptions matched by pharmacy name.
- Hospital queue, doctor management and SOS dispatch desk.
- AI companion Mevi that uses the elder's own vitals and medicines as context.
- One warm design language with a text-size control and WCAG 2.1 A/AA checks.

4.6 Final Concept

The final concept is "Care that reaches them, wherever you are." The system captures what happens in the elder's day, connects the people who can act on it, alerts the right person quickly and escalates if there is no response, and keeps a history so families and doctors can review what happened.

Chapter 5

5. Requirements Analysis

5.1 Introduction

Requirements analysis converts the problem into concrete system behaviour. ElderGuard needs both functional features and quality attributes, because a reminder or an alert that arrives late or is unreadable has failed even if it exists.

5.2 Functional Requirements

Table 5.1: Functional Requirements of ElderGuard AI

ID	Requirement	Portal
FR1	Register and sign in with email or mobile number and a role	All
FR2	Show scheduled medicines and ring a reminder at dose time until answered	Elder
FR3	Log a dose as taken and show it live to the caregiver	Elder, Caregiver
FR4	Record and chart blood pressure, sugar, oxygen, heart rate and weight	Elder, Caregiver, Doctor
FR5	Raise an SOS in one tap and notify the caregiver immediately	Elder
FR6	Escalate an unacknowledged SOS to a second contact, the hospital and emergency services	System
FR7	Acknowledge and resolve an SOS, stopping further escalation	Caregiver, Hospital
FR8	Link an elder with caregivers by elder ID	Elder, Caregiver
FR9	Book appointments and run telemedicine sessions with queue tokens	Elder, Doctor, Hospital
FR10	Write an e-prescription for a named pharmacy	Doctor
FR11	List incoming prescriptions for the matching pharmacy and approve intake	Pharmacy
FR12	Manage doctors, departments, patients and billing records	Hospital
FR13	Answer questions through an AI companion using the elder’s data	Elder
FR14	Send in-app, push and Telegram notifications	System
FR15	Provide a text-size control and keyboard-accessible interface	All

5.3 Non-Functional Requirements

Table 5.2: Non-Functional Requirements of ElderGuard AI

Attribute	Requirement
Usability	Elder text at 118% of base size; every control at least 44 px; one clear action per screen
Accessibility	WCAG 2.1 A/AA; 4.5:1 text contrast; visible 3 px focus ring; reduced-motion support
Reliability	SOS escalation driven by a scheduler sweep (30 s, 5 s in demo mode); an acknowledged SOS stops escalation
Security	bcrypt password hashing, JWT access tokens, role checks on every route
Performance	Pages load in under two seconds on a normal connection; live updates over WebSocket
Portability	Runs on a laptop with SQLite; switches to Supabase/PostgreSQL by configuration
Maintainability	Feature modules per domain; automated end-to-end and accessibility tests

5.4 Hardware Requirements

- A laptop or desktop with at least 8 GB RAM for development.
- Any modern smartphone, tablet or computer with a web browser for users.
- A stable internet connection for the AI companion, push notifications and photos.

5.5 Software Requirements

- Python 3.13 with FastAPI, SQLAlchemy and APScheduler.
- Node.js 20 or later with npm, React 18, Vite 5 and Tailwind CSS 3.
- SQLite (bundled) or a Supabase/PostgreSQL project.
- Playwright and axe-core for testing.
- Microsoft Edge, Chrome or Firefox.

5.6 User Roles and Permissions

Table 5.3: User Roles and Permissions

Role	Can view	Can change
Elder	Own medicines, vitals, appointments, reports, caregiver notes	Dose status, vitals, SOS, own profile
Caregiver	Linked elders' medicines, vitals, alerts, SOS history	Reminders, notes, SOS acknowledgement, care-team messages
Doctor	Patients linked through appointments or care	Prescriptions, reports, appointment status

Role	Can view	Can change
Hospital	Registered doctors, queue, billing, SOS dispatch	Queue entries, departments, billing records
Pharmacy	Prescriptions addressed to its own pharmacy name	Order status, inventory, deliveries

5.7 System Constraints

- Alerts are delivered in-app, by push and by Telegram. There is no SMS or voice-call provider yet.
- The hospital SOS desk receives live alerts only inside the same browser in the prototype; server push to the hospital is future work.
- The AI companion needs an internet connection and a configured API key.
- The prototype uses demo data; it has not been validated with real elders.

5.8 Requirement Summary

The requirements centre on a small number of safety-critical flows, namely reminders, SOS and the prescription handoff, supported by accessible portals and role-based access. The remaining chapters show how each requirement is met.

Chapter 6

6. Technology Stack

6.1 Introduction

The technologies were chosen so that the prototype is quick to build, easy to demonstrate on a laptop and able to grow into a hosted service.

Table 6.1: Technology Stack Used in ElderGuard AI

Layer	Technology	Purpose
Front end	React 18, Vite 5, React Router	Component-based single-page app and fast development build
Styling	Tailwind CSS 3, Atkinson Hyperlegible and Geist fonts	Design tokens, readable type
Motion	framer-motion and motion	Page transitions and gentle animation, disabled for reduced motion
Icons and charts	Phosphor Icons, Lucide, Recharts	Interface icons and vitals charts
Data fetching	TanStack Query, Axios	Cached API calls
Back end	Python, FastAPI, Uvicorn	REST API and WebSocket endpoints
Scheduling	APScheduler	Reminder sweeps and SOS escalation
Database	SQLite, Supabase/PostgreSQL	Local demo and hosted storage
Security	bcrypt, python-jose (JWT)	Password hashing and tokens
AI	Groq-hosted LLM	Mevi, the AI companion
Notifications	Firebase Cloud Messaging, Telegram	Push and chat alerts
Testing	Playwright, axe-core	End-to-end and accessibility tests

6.2 Frontend Technologies

6.2.1 React

React builds each portal from reusable components, such as the dashboard layout, the SOS card and the medicine alarm. Component reuse keeps the five portals visually consistent.

6.2.2 Vite

Vite provides a fast development server and an optimised production build. The project pre-bundles its main dependencies so that the development server starts reliably.

6.2.3 Tailwind CSS

Tailwind supplies the design tokens, such as the single teal accent, the warm paper background and the 12 px and 24 px radii. A per-role root font scale makes the elder portal larger and the clinical portals denser.

6.2.4 Motion libraries

framer-motion provides spring page transitions, staggered card entrances and a sliding navigation pill. All motion honours the operating system's reduced-motion setting.

6.3 Backend Technologies

6.3.1 Python and FastAPI

FastAPI exposes more than a hundred REST endpoints grouped by domain, with automatic validation and interactive documentation. A WebSocket endpoint streams live changes to the elder and caregiver.

6.3.2 APScheduler

APScheduler runs background sweeps: medicine reminder sessions every 15 to 60 seconds and the SOS escalation sweep every 30 seconds (5 seconds in demo mode).

6.4 Database Technology

The prototype uses SQLite for a zero-setup local database. When Supabase credentials are configured, the same data model is stored in hosted PostgreSQL. The data layer reads and writes through one helper so that both modes behave the same.

6.5 Artificial Intelligence

The companion, Mevi, calls a Groq-hosted language model. The prompt includes the elder's own medicines and recent vitals so answers are specific, and instructs the model to avoid diagnosis and to suggest contacting a doctor or emergency services when appropriate.

6.6 Notification Technology

Notifications are stored in the database, pushed to the browser through WebSocket, and sent through Firebase Cloud Messaging and Telegram when configured.

6.7 Technology Integration

The React front end calls the FastAPI back end over HTTP and WebSocket. The back end reads and writes the database, runs scheduled jobs, calls the language model and sends notifications.

6.8 Technology Selection

These technologies are widely used, well documented, free to run locally and able to scale to a hosted deployment. They also allowed a small team to build five portals and a real-time safety feature within a semester.

Chapter 7

7. System Design

7.1 System Architecture

ElderGuard follows a layered architecture. Five portals in the browser talk to a FastAPI service, which uses background jobs, a WebSocket hub, an AI companion and an OCR component, and stores data in SQLite or Supabase/PostgreSQL. Figure 7.1 shows the layers.

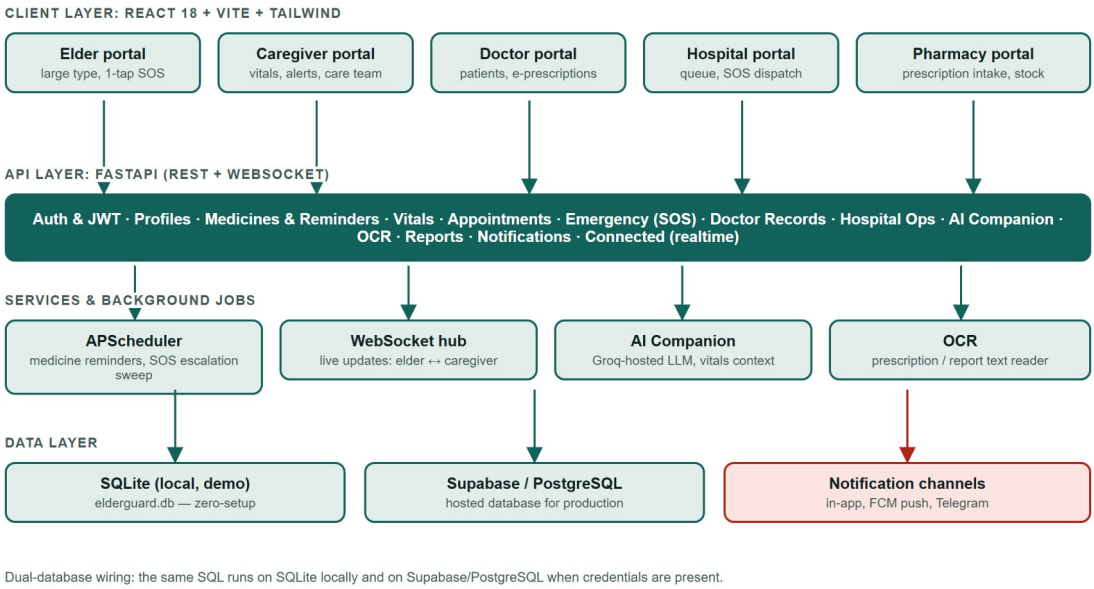


Figure 7.1: System architecture of ElderGuard AI

7.2 Introduction

The design separates what users see (portals), what the system does (API modules) and what must happen without anyone clicking (scheduled jobs), so that safety-critical behaviour does not depend on a screen being open.

7.3 Working Flow

A user signs in with an email or mobile number and a role. The server verifies the password hash and returns a signed token, and the front end opens the portal for that role. From then on every request carries the token. Medicine reminders are created by a

scheduler; the elder answers them in the portal; the caregiver's dashboard is refreshed through WebSocket. An SOS creates an event, notifies the first tier, and the escalation sweep advances the event until someone acknowledges it.

7.4 System Components

7.4.1 Authentication Module

Handles registration, sign-in, bcrypt password hashing, JWT access and refresh tokens, and role checks on protected routes.

7.4.2 Medicine and Reminder Module

Stores medicines, generates reminder sessions, rings the elder's alarm, records Taken or missed, and tracks stock.

7.4.3 Vitals Module

Records readings, applies target ranges and provides trend data for charts.

7.4.4 Emergency (SOS) Module

Creates SOS events, runs the tiered escalation, records every action in a log and supports acknowledgement and resolution.

7.4.5 Clinical Records Module

Holds appointments, telemedicine sessions, doctor prescriptions and reports for the doctor portal.

7.4.6 Hospital and Pharmacy Modules

Hospital operations (queue, departments, billing) and pharmacy intake (prescriptions routed by pharmacy name).

7.4.7 AI Companion and Notification Modules

The companion builds a prompt from the elder's data and calls the language model. The notification module stores alerts and sends them over the available channels.

7.5 Use Case Diagram

Figure 7.2 lists the actions available to each actor.

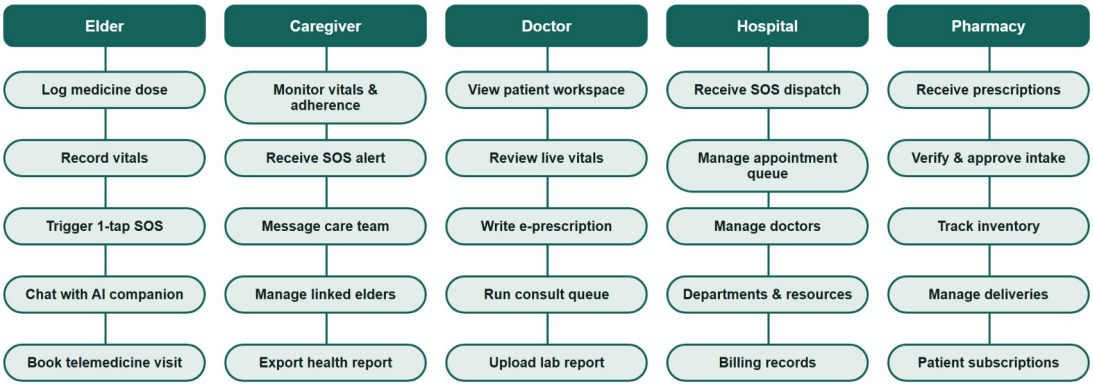


Figure 7.2: Use cases by role

7.6 SOS Escalation Flow

The default escalation policy has four tiers. Tier 1 notifies the caregiver immediately. Tier 2 notifies the secondary contact after 2 minutes, tier 3 notifies the hospital after 5 minutes and tier 4 contacts emergency services after 10 minutes, unless the event has been acknowledged. A sweep job checks events every 30 seconds. When the environment variable SOS_DEMO_SECONDS is set, the delays become multiples of that value so that a presenter can show the whole chain quickly.

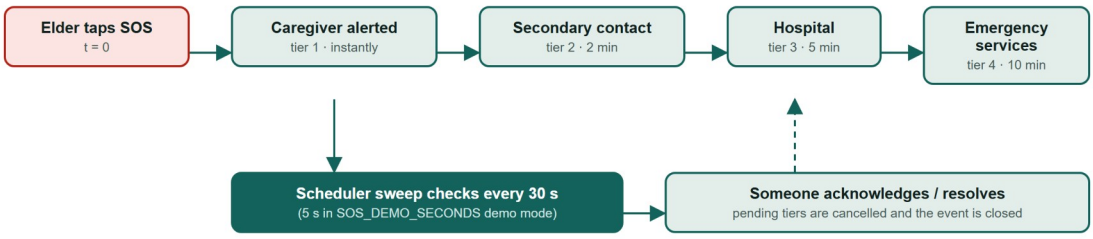


Figure 7.3: SOS escalation workflow

7.7 Prescription Handoff Flow

A doctor writes an e-prescription and names a pharmacy. The prescription is stored with status Active (e-Signed). The pharmacy portal lists prescriptions whose pharmacy name matches the pharmacy's own profile name, ignoring case, and the pharmacist can verify and approve intake. The same prescription also appears in the elder's medicine list.

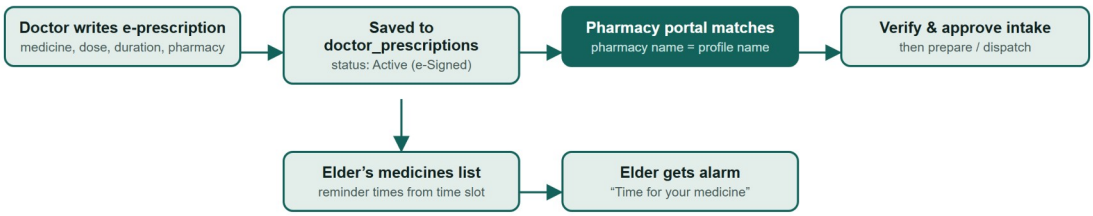


Figure 7.4: Doctor-to-pharmacy prescription handoff

7.8 Data Flow

Figure 7.5 shows the data flow. Users are authenticated first. Health tracking and clinical activity are written to the database. SOS events are sent straight to the notification process, and the notification process alerts the caregiver, the hospital and the pharmacy.

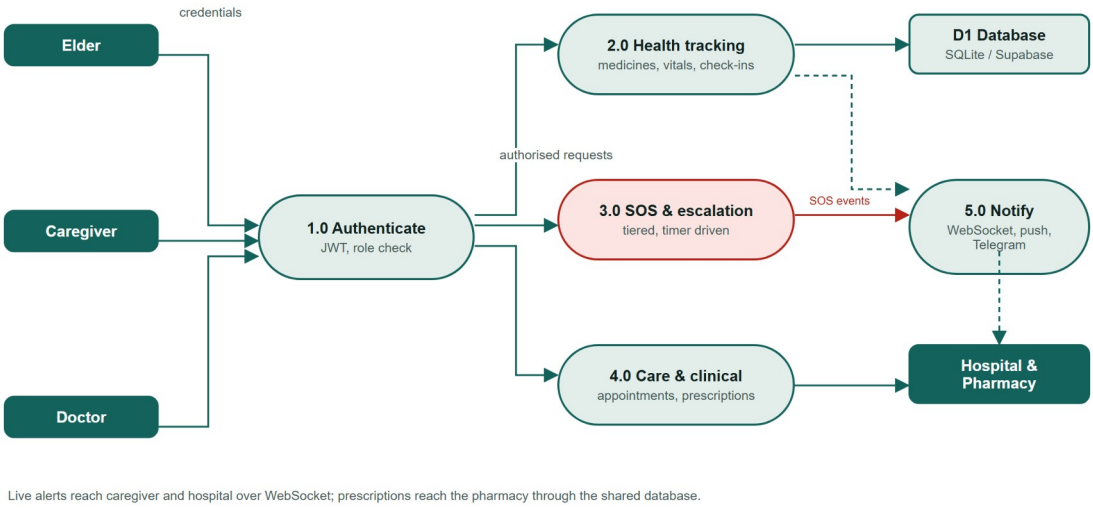


Figure 7.5: Data flow diagram of ElderGuard AI

7.9 System Design Considerations

- **Safety first:** SOS must not depend on a screen being open, so escalation runs in a back-end sweep job.
- **Graceful degradation:** every external channel (push, Telegram, language model) is optional, and failure of one does not stop the others.
- **Least privilege:** each role can read and change only what Table 5.3 allows.
- **Honest scope:** the interface says what it does. It does not claim SMS, calls or ambulance dispatch that are not implemented.

Chapter 8

8. Database Design

8.1 Introduction

The database stores users, role profiles, medicines and dose logs, vitals, appointments, prescriptions, SOS events and notifications. The same schema runs on SQLite locally and on PostgreSQL through Supabase.

8.2 Database Objectives

- Keep one record per elder that every authorised role can read.
- Make every SOS action traceable.
- Keep roles separate but linked through identifiers.
- Allow the schema to be created automatically on first start.

8.3 Main Database Entities

The principal entities are users, elder_profiles, caregiver_profiles, doctor_profiles, hospital_profiles, pharmacy_profiles, elder_caregiver_links, connected_medicines, medicine_logs, connected_vitals, appointments, doctor_prescriptions, connected_sos_events, hospital_queue_entries, hospital_billing_records and connected_notifications.

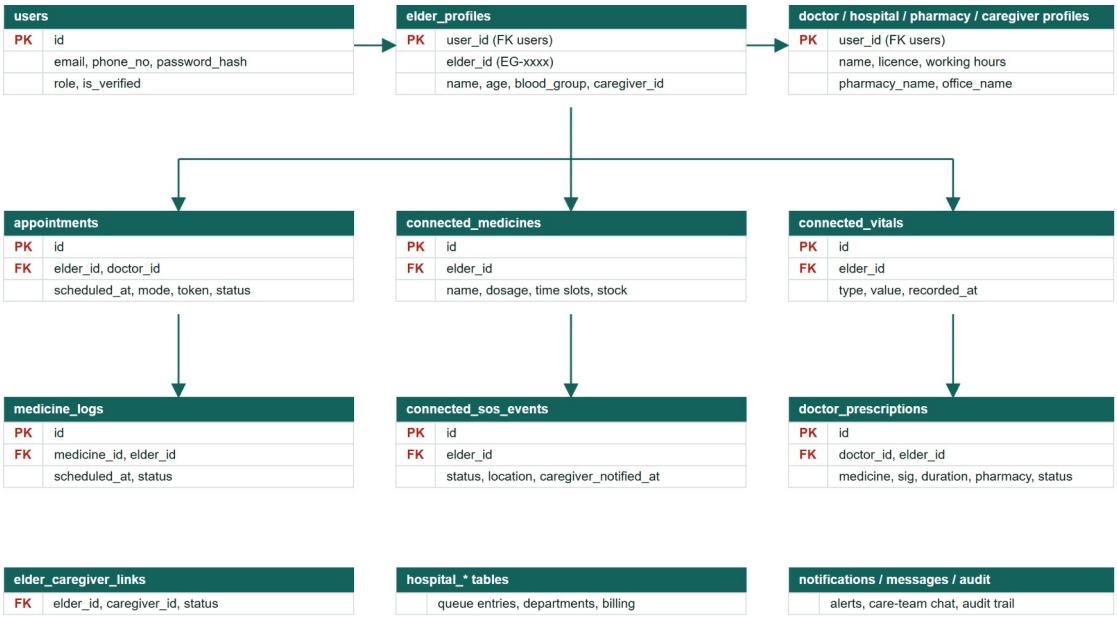


Figure 8.1: Entity relationship diagram of ElderGuard AI

8.4 User Table

Table 8.1: Users Table

Column	Type	Description
id	TEXT (PK)	Unique user identifier
email, phone_no	TEXT, unique	Sign-in identifiers
password_hash	TEXT	bcrypt hash of the password
auth_provider	TEXT	Local or Google
role	TEXT	elder, caregiver, doctor, hospital or pharmacy
is_verified	BOOLEAN	Account verification flag
created_at, updated_at	TEXT	Timestamps

8.5 Elder Profile Table

Table 8.2: Elder Profiles Table

Column	Type	Description
user_id	TEXT (PK, FK)	Links to users.id
elder_id	TEXT, unique	Shareable code such as EG-1A2B3C4D used to link caregivers
name, age_or_dob, gender	TEXT	Personal details
blood_group, primary_language	TEXT	Clinical and language preferences
home_address, phone_no	TEXT	Contact details
caregiver_id	TEXT	Primary linked caregiver

8.6 Prescription Table

Table 8.3: Doctor Prescriptions Table

Column	Type	Description
id	TEXT (PK)	Prescription identifier
doctor_id, elder_id	TEXT (FK)	Prescriber and patient
medicine, sig	TEXT	Medicine name and dosage instruction
duration, refills	TEXT, INTEGER	Course length and refills
pharmacy	TEXT	Name of the destination pharmacy
status	TEXT	For example Active (e-Signed)
time_slot, meal_timing	TEXT	Used to build reminder times

8.7 SOS Event Table

Table 8.4: SOS Events Table

Column	Type	Description
id	TEXT (PK)	Event identifier
elder_id	TEXT (FK)	Elder who raised the SOS
triggered_at	TEXT	Time the SOS was raised
location	TEXT	Location text if available
status	TEXT	active, acknowledged or resolved
caregiver_notified_at	TEXT	Time tier 1 was notified
resolved_by, resolution_notes	TEXT	Who closed it and why

8.8 Appointments, Medicines and Vitals

The appointments table records the elder, doctor, scheduled time, mode (telemedicine or in person), queue token and status. connected_medicines stores name, dosage, time slots and stock; medicine_logs stores each scheduled dose and its status. connected_vitals stores the type, value and recording time of every reading.

8.9 Database Relationships

- One user has one role profile.
- One elder has many medicines, vitals, appointments, prescriptions and SOS events.
- One elder can be linked to many caregivers, and one caregiver to many elders, through elder_caregiver_links.
- A prescription belongs to one doctor and one elder and is addressed to one pharmacy by name.

8.10 Database Security

Passwords are stored only as bcrypt hashes. Every protected route checks the token and role before touching data, and caregivers and doctors see an elder only through a link or an appointment. Caregiver actions are written to an audit log.

8.11 Database Summary

The schema is small and normalised enough to be understood quickly, and the elder identifier ties all clinical and safety data together.

Chapter 9

9. Module Description

9.1 Introduction

This chapter describes each module from the user's point of view, with the portal screens that implement it.

Table 9.1: ElderGuard Module Description

ID	Module	Function
M1	Authentication	Register, sign in, role selection
M2	Medicine reminders	Schedule, alarm, Taken action, stock
M3	Vitals	Log, chart and flag readings
M4	SOS escalation	One-tap alert and four-tier escalation
M5	Appointments and telemedicine	Booking, queue tokens, consult room
M6	Doctor workspace	Patients, vitals, prescriptions, reports
M7	Pharmacy	Prescription intake, orders, inventory, delivery
M8	Hospital	Queue, doctors, departments, billing, SOS dispatch
M9	AI companion	Mevi chat with the elder's data as context
M10	Notifications	In-app, push and Telegram alerts
M11	Real-time hub	WebSocket updates for elder and caregiver

9.2 Authentication Module

The sign-in screen lets the user choose a role, enter an email address or mobile number and a password, and open the correct portal. Registration collects the details needed for that role; an elder receives a shareable elder ID.

9.3 Elder Portal

The dashboard greets the elder by name and shows today's medicines, a large red one-tap SOS card, a daily "I'm OK" check-in, a short health report from Mevi, the latest vitals and notes from the caregiver and doctor. When a dose is due, an alarm dialog appears with a Medicine Taken button.

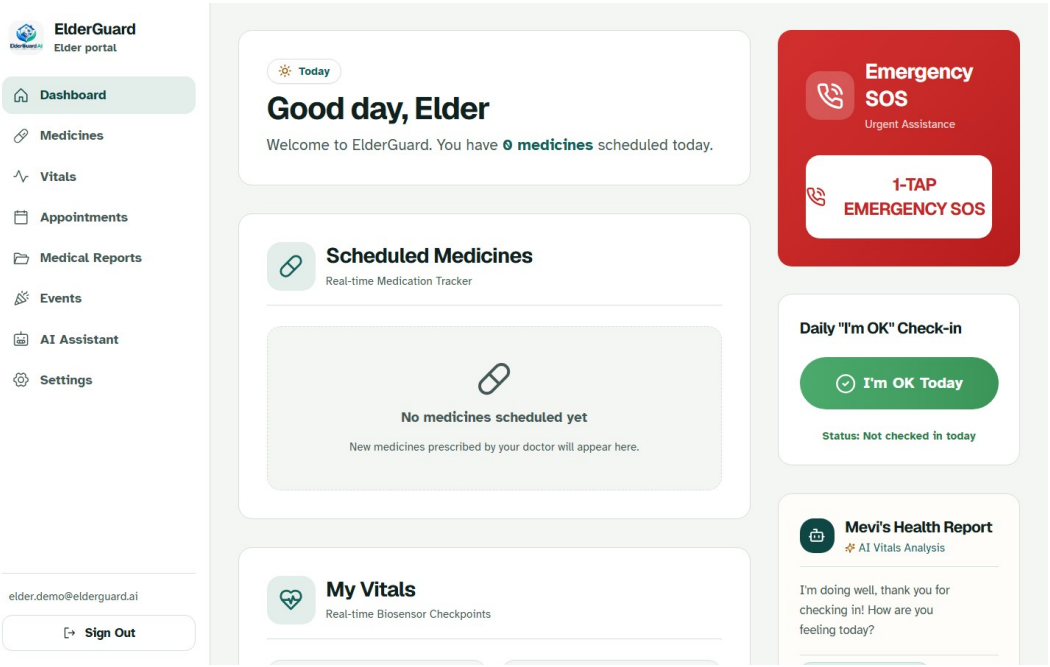


Figure 9.1: Elder dashboard

9.4 Medicine Reminder Module

Medicines are listed by time of day. A scheduler creates reminder sessions, starts the ring, and times out unanswered doses so the caregiver can be told. The elder can test the alarm sound and mark each dose as taken.

9.5 Vitals Module

The elder can type or speak a reading. The caregiver and doctor see charts and target ranges, and out-of-range readings are flagged as anomalies.

9.6 SOS Escalation Module

The SOS card sends one request. The server creates an event and notifies tier 1; the scheduler advances the escalation. Caregiver and hospital portals show active alerts until they are acknowledged or resolved.

9.7 Caregiver Portal

The caregiver chooses a linked elder and sees live vitals, medicine adherence, today's dose stream, quick actions, vitals trends, alerts and the care-team thread.

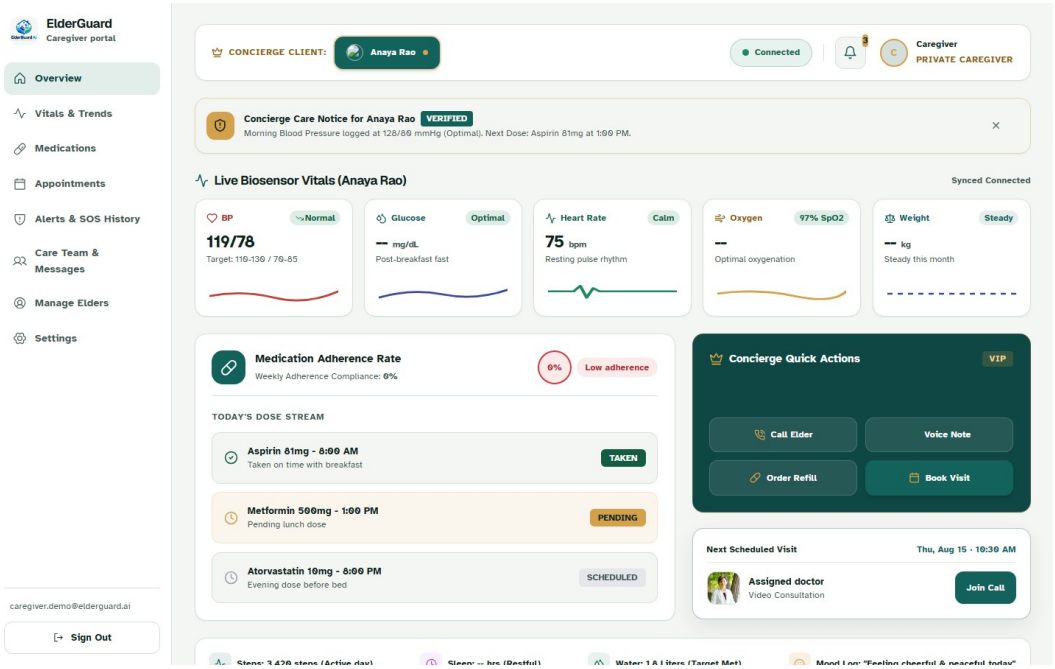


Figure 9.2: Caregiver dashboard

9.8 Doctor Portal

The doctor sees today's consultation queue, patient roster and a connected patient workspace with live vitals, prescriptions and reports, and can write a prescription for a named pharmacy.

9.9 Pharmacy Portal

Incoming prescriptions from doctors appear in an intake queue; the pharmacist verifies and approves intake, and then tracks orders, inventory and deliveries.

9.10 Hospital Portal

The hospital desk manages doctors, the appointment queue, departments and billing, and has an Emergency and SOS Dispatch page for live alerts.

9.11 AI Companion Module

Mevi answers questions about routines and medicines in plain language, uses the elder's data for context, and avoids diagnosis.

9.12 Module Integration

All modules share the elder identifier and the same database, so an action in one portal is visible in another. For example, a prescription written in the doctor portal appears in the pharmacy queue and in the elder’s medicine list.

Chapter 10

10. Implementation and Working Principle

10.1 Introduction

This chapter explains how the front end, back end, scheduler and database work together to deliver the safety-critical behaviour.

10.2 Frontend Implementation

The front end is a single React application. A shared dashboard layout provides a light sidebar with one teal accent, a bottom tab bar on phones, a skip link and page transitions. Each role sets a different root font scale: 118% for the elder, 95% for the caregiver and 93.75% for the clinical portals. The design uses the Atkinson Hyperlegible typeface for body text, a 12 px text floor, 44 px touch targets and a 3 px focus ring. Motion is added through framer-motion and removed for users who prefer reduced motion.

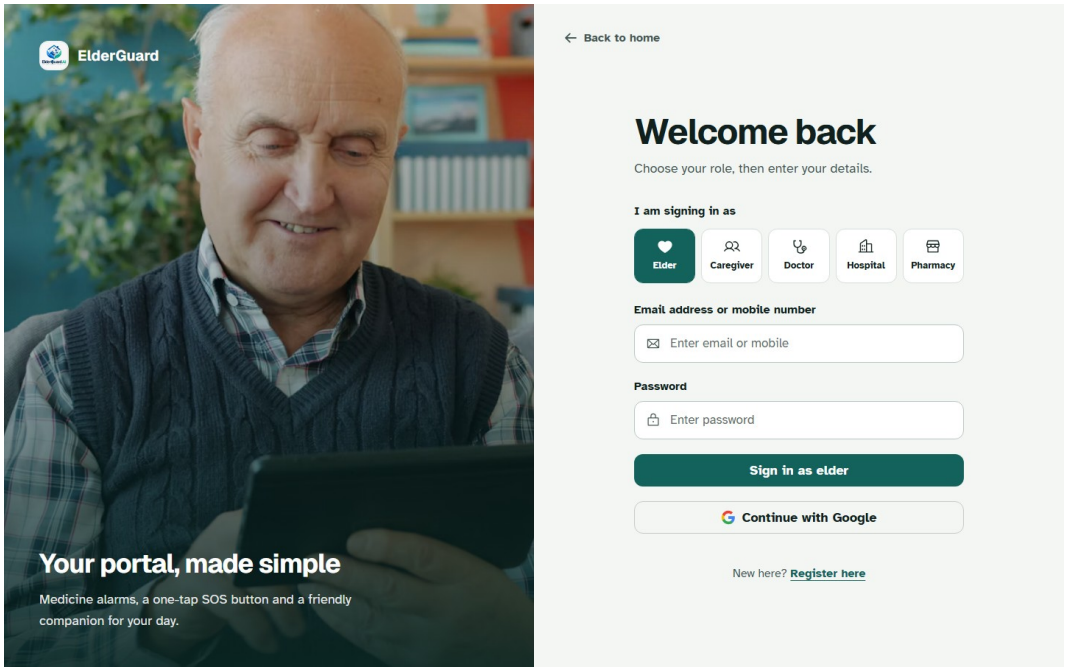


Figure 10.1: Role-based sign-in screen

10.3 Backend Implementation

The FastAPI application includes routers for authentication, profiles, medicines, reminders, vitals, appointments, emergency, doctor records, hospital operations, AI companion,

notifications and the realtime connected hub. Tables are created on start-up if they do not exist, so a new developer only needs to run the server.

10.4 Reminder Processing

A reminder sweep creates sessions for upcoming doses, a second sweep starts the ring and a third checks time-outs. When the elder presses Medicine Taken, the dose is logged and pushed to the caregiver over WebSocket.

10.5 SOS Processing

When the elder presses SOS, trigger_sos writes an event and notifies tier 1. The escalation sweep compares the elapsed time with each tier's delay and notifies the next recipient if the event is still unacknowledged. Every action is written to a log. Acknowledging or resolving the event stops further escalation.

```
# Default escalation policy (backend/app/emergency/service.py)
tiers = [
  {"tier_number": 1, "recipient_type": "caregiver",           "timeout_minutes": 0},
  {"tier_number": 2, "recipient_type": "secondary_caregiver", "timeout_minutes": 2},
  {"tier_number": 3, "recipient_type": "hospital",            "timeout_minutes": 5},
  {"tier_number": 4, "recipient_type": "emergency_services",  "timeout_minutes": 10},
]
if SOS_DEMO_SECONDS > 0:           # demo mode compresses the timeline
    for tier in tiers:
        tier["delay_seconds"] = (tier["tier_number"] - 1) * SOS_DEMO_SECONDS
```

10.6 Prescription Handoff

When a doctor creates a prescription the server stores the pharmacy name entered by the doctor. When the pharmacy opens its queue the server reads the pharmacy's profile name, converts it to lower case and lists prescriptions whose pharmacy name matches. This avoids needing a pharmacy to be registered before a doctor can prescribe, at the cost of requiring the names to match.

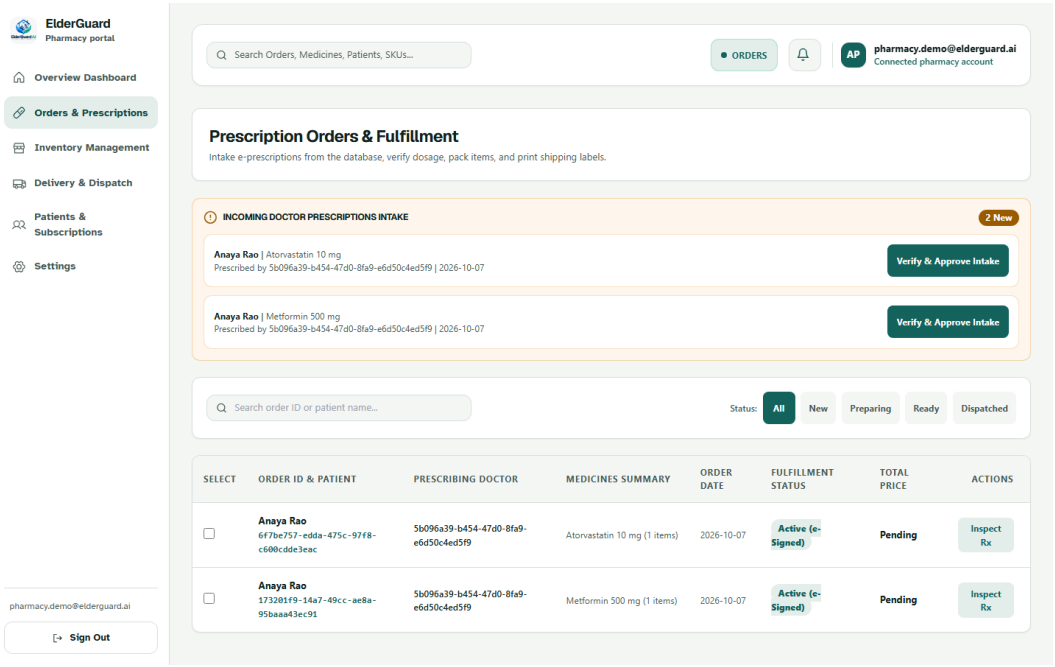


Figure 10.2: Pharmacy queue showing prescriptions received from the doctor

10.7 Demo Data and Demo Mode

A seed script creates the five demo accounts and a connected story: an elder, her caregiver, a doctor with a telemedicine appointment, two prescriptions addressed to a named pharmacy, a hospital and the pharmacy. Starting the application with the demo script runs the SOS escalation in seconds instead of minutes, so the whole chain can be shown during a presentation.

10.8 AI Companion

The route builds a system prompt containing the elder's recent vitals and medicines and sends the conversation to the language model. The model is told to speak simply, to avoid diagnosis and to point the elder to the caregiver, doctor or emergency services when needed.

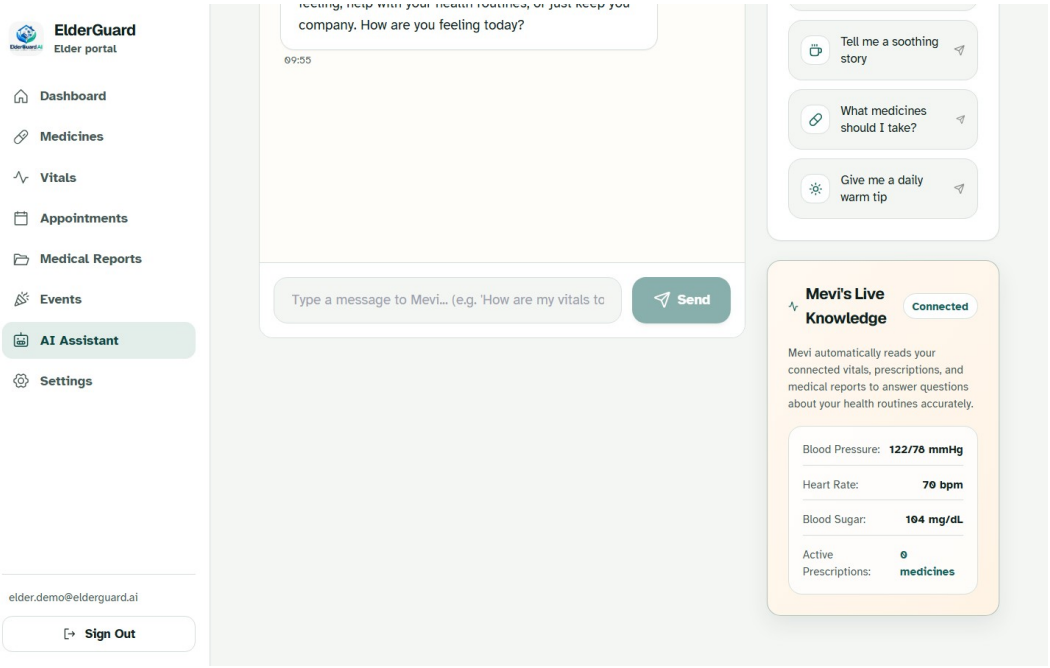


Figure 10.3: AI companion screen

10.9 Implementation Summary

Safety-critical logic runs in the back end and does not depend on a screen being open. The portals present the same underlying record in a form suited to each role.

Chapter 11

11. Testing

11.1 Introduction

Testing combined automated browser tests, an automated accessibility audit, a timed SOS drill and manual checks. Automated tests were written with Playwright and axe-core and live in frontend/tests.

11.2 Objectives of Testing

- Confirm that every portal page opens without an error screen.
- Confirm the demo story works from doctor to pharmacy.
- Detect accessibility problems against WCAG 2.1 A/AA.
- Confirm that the SOS escalation fires every tier in order.
- Confirm the layouts work at phone size.

11.3 Types of Testing

11.3.1 End-to-end Testing

An automated browser signs in through the real form as each role, opens every navigation item and checks that the page loads and that no uncaught error occurs. The suite has 22 tests covering the five portals and phone size.

11.3.2 Accessibility Testing

The axe-core engine scans every page against WCAG 2.1 A/AA rules. The scan runs with reduced motion so that animation does not affect colour contrast readings. The result is no serious or critical violations.

11.3.3 Story Testing

Story tests follow the demo narrative: the doctor has the elder on the roster, prescriptions written by the doctor reach the pharmacy queue, pharmacy screens show real data or honest empty states, and the landing page describes escalation the way the system really behaves.

11.3.4 Mobile Testing

The suite is also run at 375 × 812 pixels to check the bottom tab bar, the More menu and that content is not clipped.

11.3.5 SOS Escalation Drill

With SOS_DEMO_SECONDS set, an SOS was raised and left unacknowledged. All four tiers fired in order within the compressed timeline.

11.3.6 Backend Tests

The backend folder contains scripts that exercise the emergency, appointment, vitals, medicine reminder, notification, OCR, report, analytics and connected-portal services.

11.4 Functional Test Cases

Table 11.1: Functional Test Cases

ID	Test case	Expected result	Result
T1	Sign in as each of the five roles	Correct portal opens	Pass
T2	Open every sidebar item for every role	No error screen	Pass
T3	Elder portal at desktop width	No sideways scrolling	Pass
T4	Doctor prescription for GreenLeaf Pharmacy	Appears in pharmacy queue	Pass
T5	Tiered SOS left unanswered in demo mode	Tiers 1 to 4 fire in order	Pass
T6	Accessibility scan of every page	No serious violations	Pass
T7	Phone-size layout	Bottom tab bar and More menu work	Pass
T8	Wrong password; sign out	Visible alert; return to landing page	Pass

11.5 Defects Found and Fixed

Table 11.2: Notable Defects Found During Testing

Defect	Fix
Intermittent error screen on first load after dependency discovery by the dev server	Pre-bundled key packages in Vite and added a one-time self-healing reload in the error boundary
A seeded doctor profile with working hours stored as text crashed the profile page	Normalised working hours in the API and the front end
Alarm dialog blocked the page during automated tests	Tests dismiss the overlay through the DOM

Defect	Fix
Low-contrast badge and unlabelled bell icon found by axe	Colours darkened and an accessible label added

11.6 Testing Result

At the time of writing, all 22 automated end-to-end tests pass, the accessibility audit reports no serious violations and the SOS drill fired all four tiers. These figures describe the prototype on demo data. They do not show that the product helps real elders, which needs a pilot.

11.7 Conclusion of Testing

The automated checks give confidence that the prototype works across all portals and is accessible. Usability sessions with older adults are the most important remaining test.

Chapter 12

12. Project Evaluation

12.1 Introduction

This chapter evaluates the prototype against its objectives, honestly separating what was measured from what is still a target.

12.2 Objective Evaluation

Table 12.1: Project Objective Evaluation

Objective	Status	Evidence
Reminder that is answered and visible to family	Met in prototype	Alarm dialog, Taken action, caregiver dose stream
One-tap SOS with escalation	Met in prototype	Four-tier drill; in-app and push alerts
Vitals tracking	Met	Typed and spoken entry, charts, target ranges
Doctor-to-pharmacy handoff	Met	Seeded prescriptions appear in the pharmacy queue
AI companion	Met	Mevi with vitals context
Accessible design	Met on automated rules	axe: no serious violations; manual elder testing pending
Role-based access	Partly met	Role checks in place; consent and audit to be extended

12.3 Usability Evaluation

The interface uses very large type for elders, a text-size control, one primary action per screen and a single colour accent. These choices follow accessibility guidance and the empathy findings. Real usability testing with older adults has not yet been done.

12.4 Performance and Reliability Evaluation

Escalation is driven by a back-end sweep that does not depend on any open page. In the prototype the sweep runs inside the API process, so a restart would pause it. A production system needs a durable job queue.

12.5 Security Evaluation

Passwords are hashed with bcrypt and routes check tokens and roles. Before a pilot, the project needs consent management, encrypted secrets, audit trails and review against India's Digital Personal Data Protection Act.

12.6 Advantages

- One shared record for the whole care circle.
- Safety features that do not need the elder to be able to phone anyone.
- Designed for older users and tested for accessibility.
- Runs on a laptop with no external services, and scales to a hosted database.

12.7 Limitations

- No SMS or voice-call provider; alerts are in-app, push and Telegram.
- The hospital desk receives SOS alerts through a same-browser broadcast in the prototype, not through server push.
- Pharmacy order status changes are held in the browser only.
- The prescription scanner is a prototype screen without a production OCR service.
- Verified on demo data only; no real elders took part.

12.8 Overall Evaluation

ElderGuard meets its prototype objectives and demonstrates a complete, connected care flow. It is a strong foundation for a pilot, and the limits above define the next work.

Chapter 13

13. Conclusion and Future Enhancements

13.1 Conclusion

ElderGuard AI shows that elder care becomes simpler and safer when the elder, family, doctor, pharmacy and hospital share one live record. The prototype delivers answerable medicine reminders, a one-tap SOS with tiered escalation, vitals tracking, a doctor-to-pharmacy prescription flow and an AI companion, in five portals designed for their users. It is verified by 22 automated end-to-end tests, an accessibility audit with no serious violations and a timed SOS drill. The work also made clear that technology alone is not enough: the next step is to learn from real elders and families.

13.2 Future Enhancements

Table 13.1: Roadmap

Phase	Enhancement	Benefit
1	SMS and voice-call alerts with delivery receipts	Reach people without the app
1	Server push to the hospital SOS desk and a durable job queue	Reliable escalation
1	Verified doctor and hospital accounts	Trusted prescribers
2	Multilingual voice assistant (Tamil, Hindi and others)	Easier for elders
2	Installable mobile app with offline mode	Always-ready SOS
3	Wearable integration and fall detection	Automatic alerts
3	ABDM and EHR integration with consent	Continuity of records
Pilot	Usability sessions and a clinic or care-home pilot	Measured impact (target: 90% or more doses on time)

Appendix A – Results and Screenshots

A.1 Introduction

The following screens were captured from the running prototype with the seeded demo data.

A.2 Landing and Sign-in

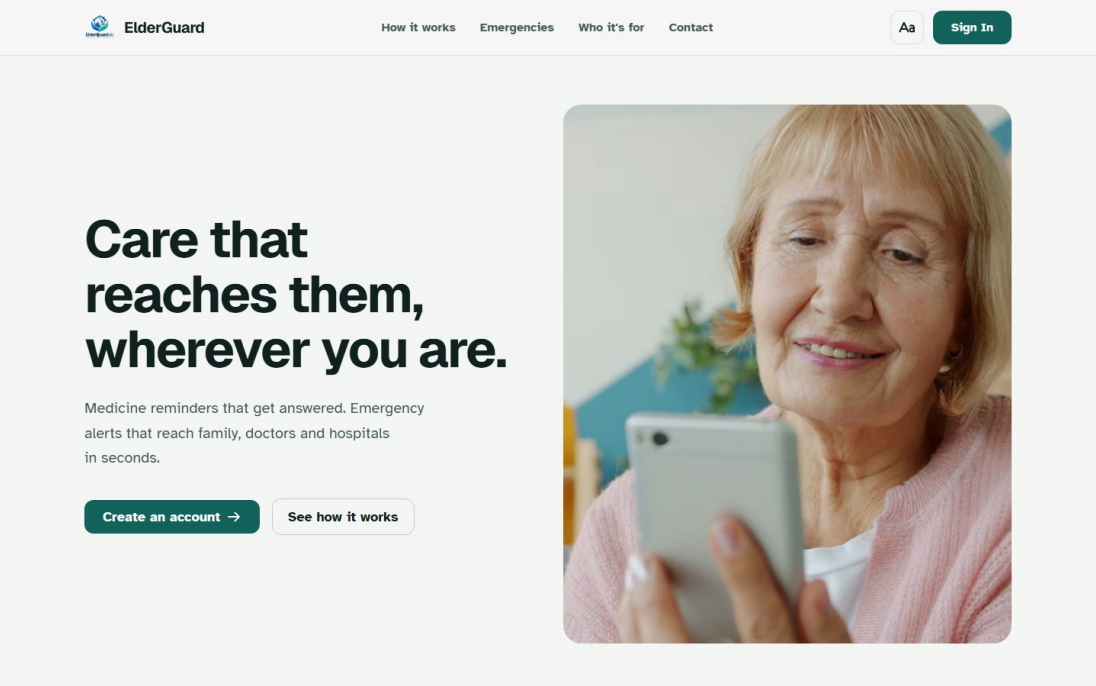


Figure A.1: Landing page

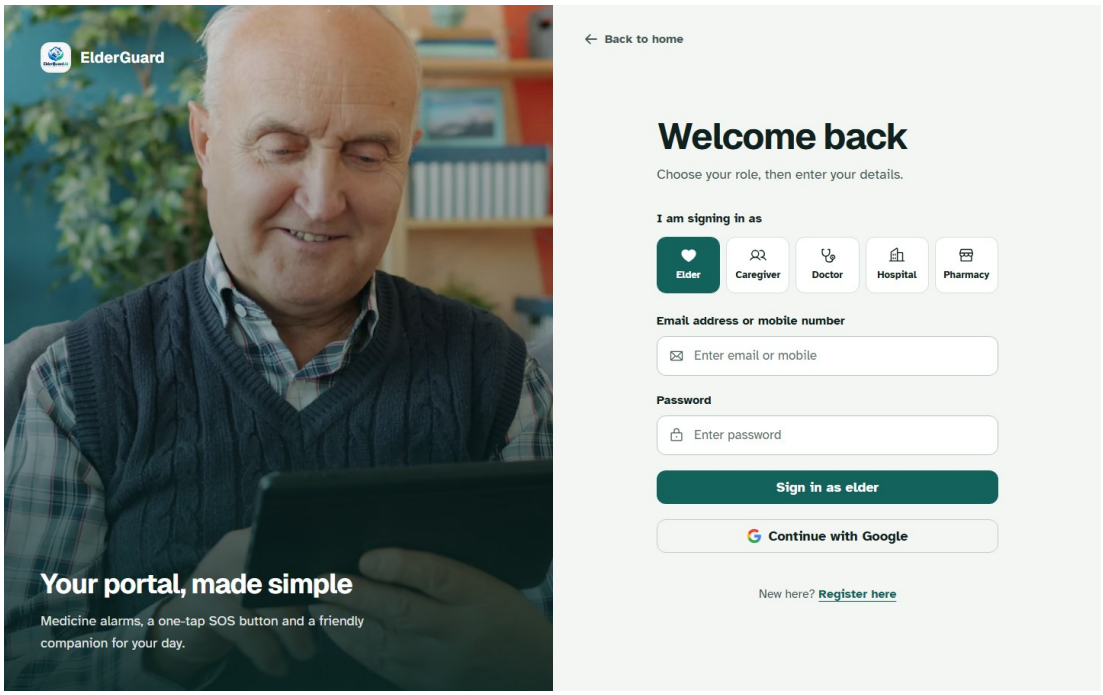


Figure A.2: Sign-in screen

A.3 Elder Portal

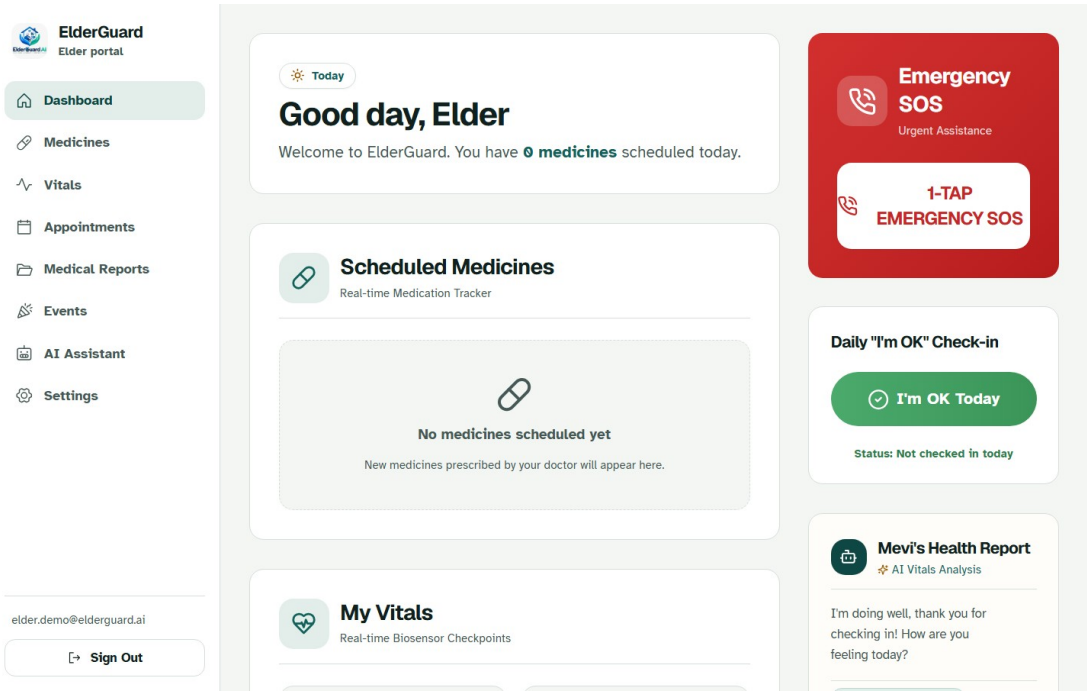


Figure A.3: Elder dashboard

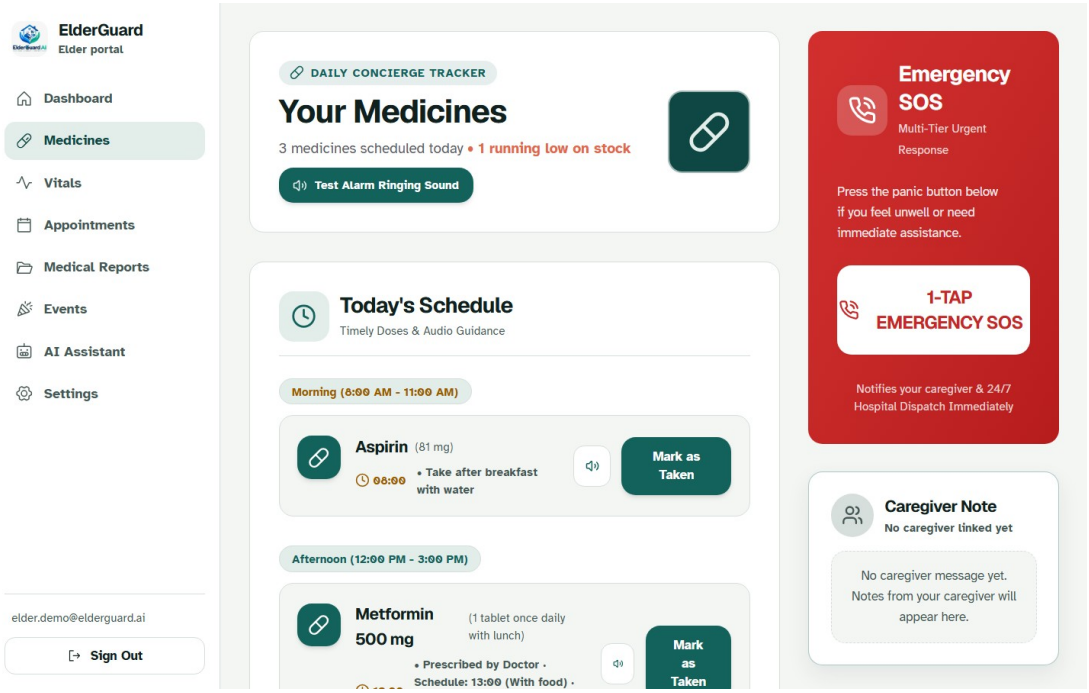


Figure A.4: Elder medicines

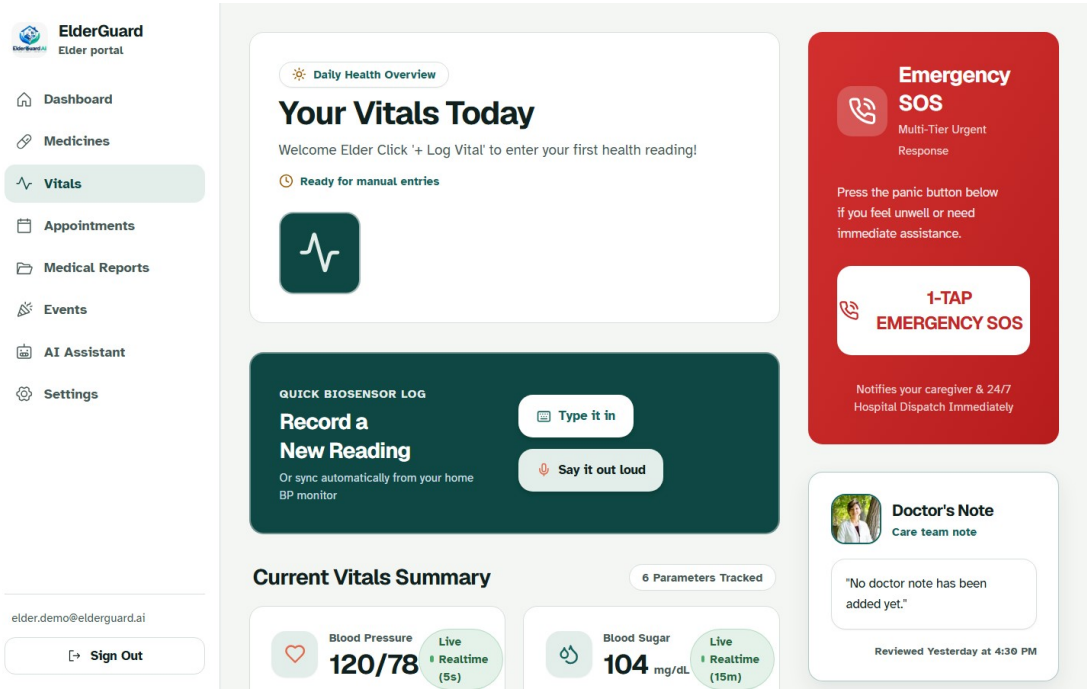


Figure A.5: Elder vitals

A.4 Caregiver Portal

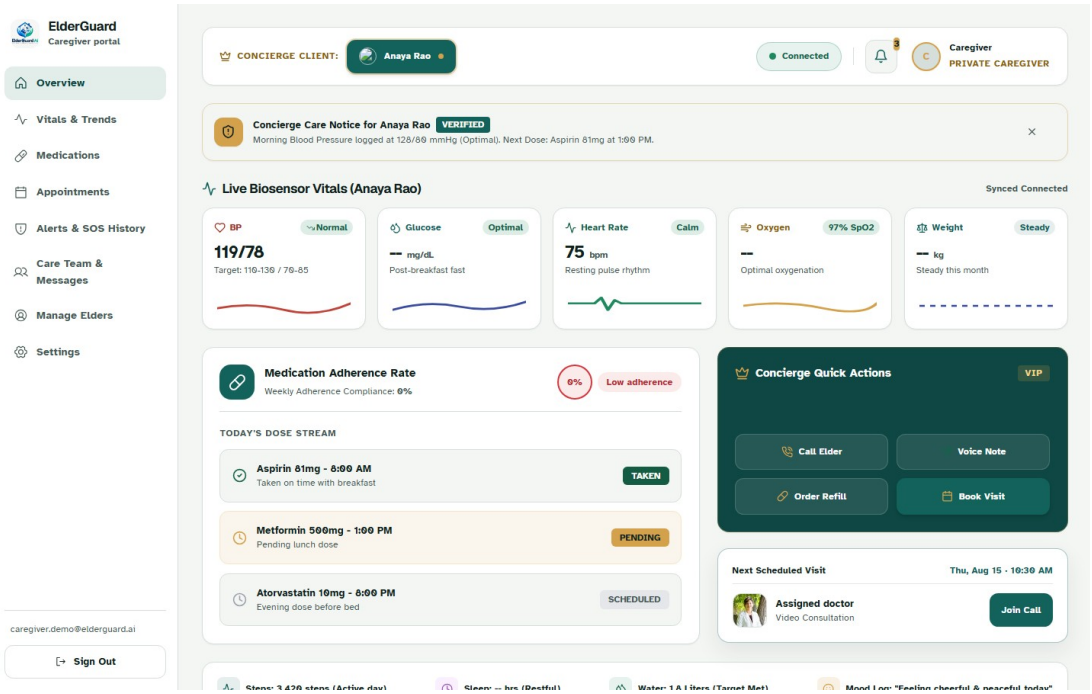


Figure A.6: Caregiver overview

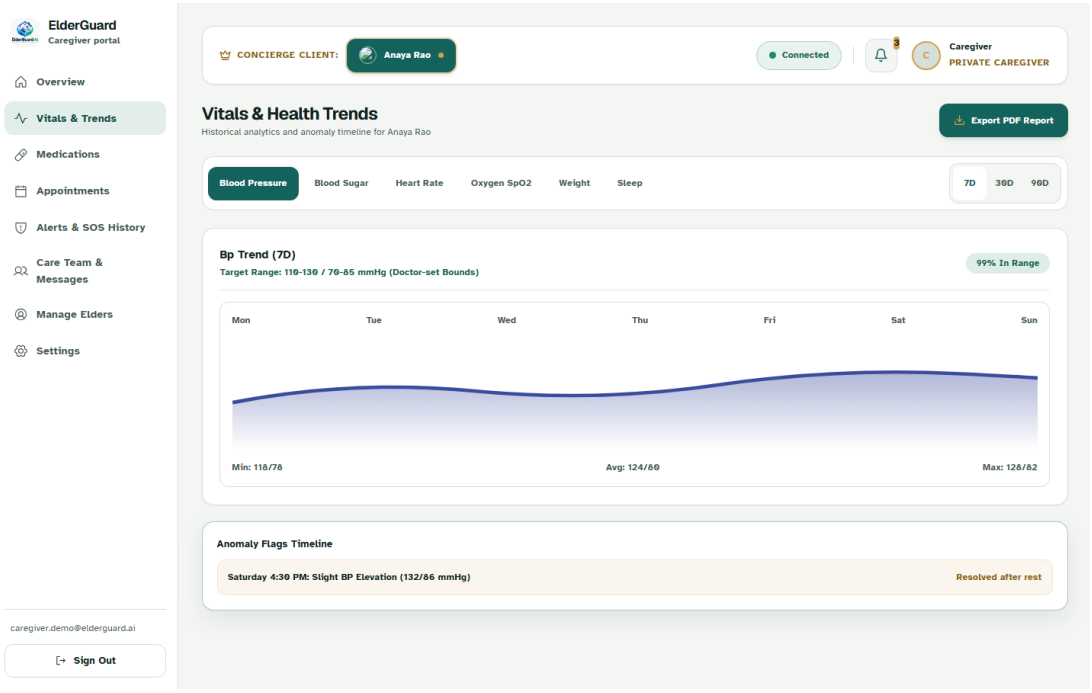


Figure A.7: Vitals and health trends

A.5 Doctor Portal

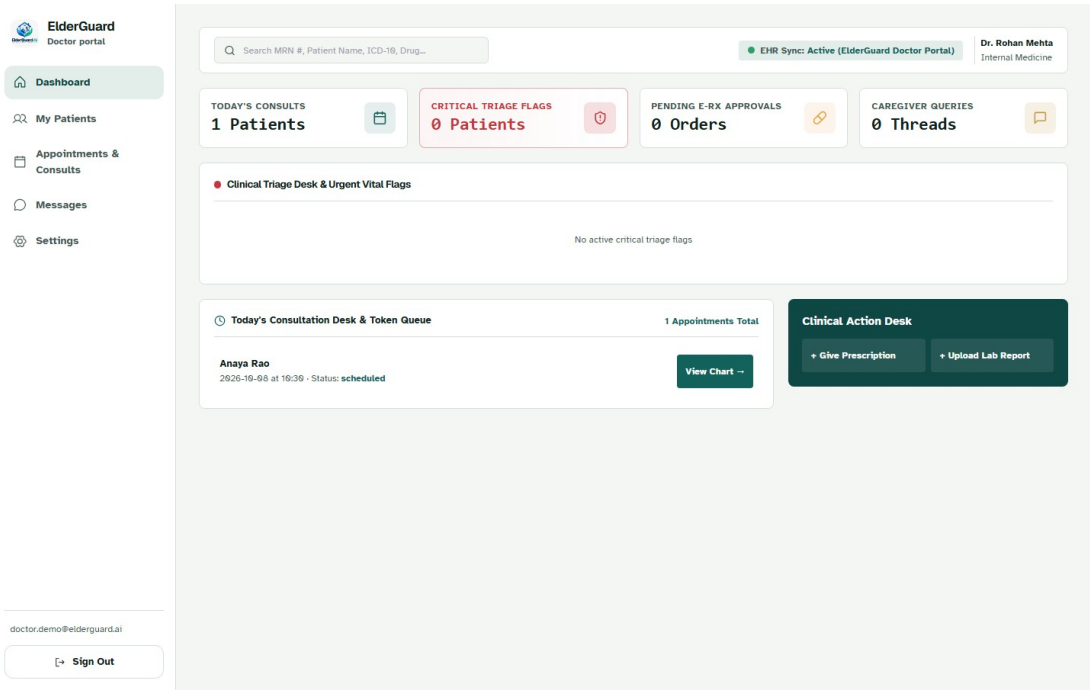


Figure A.8: Doctor dashboard

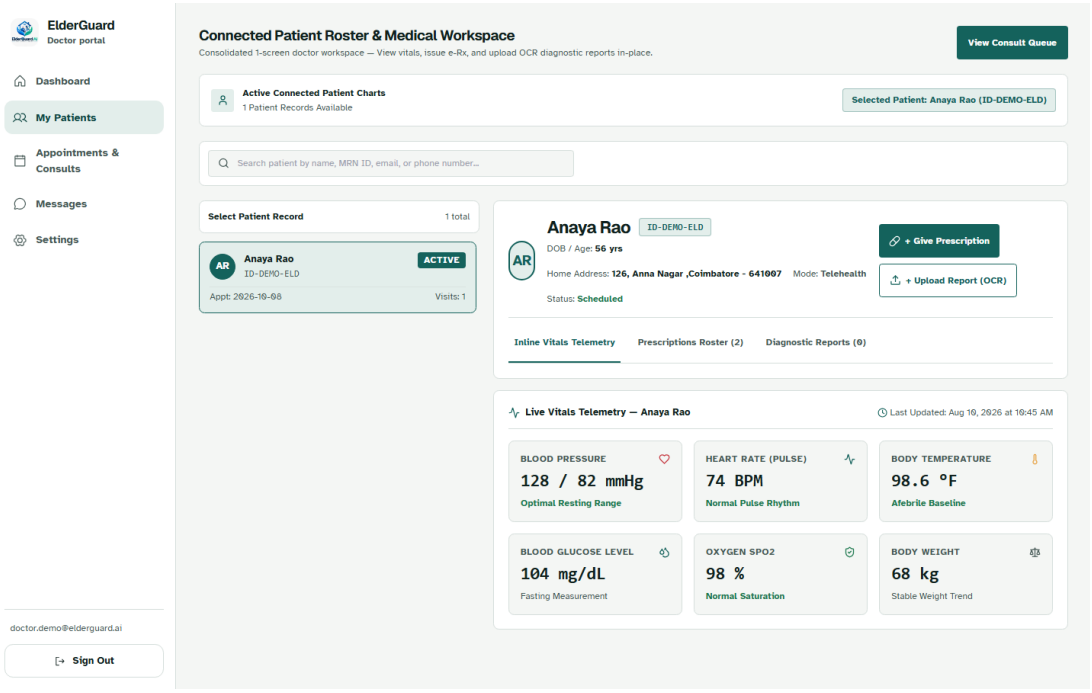


Figure A.9: Connected patient workspace

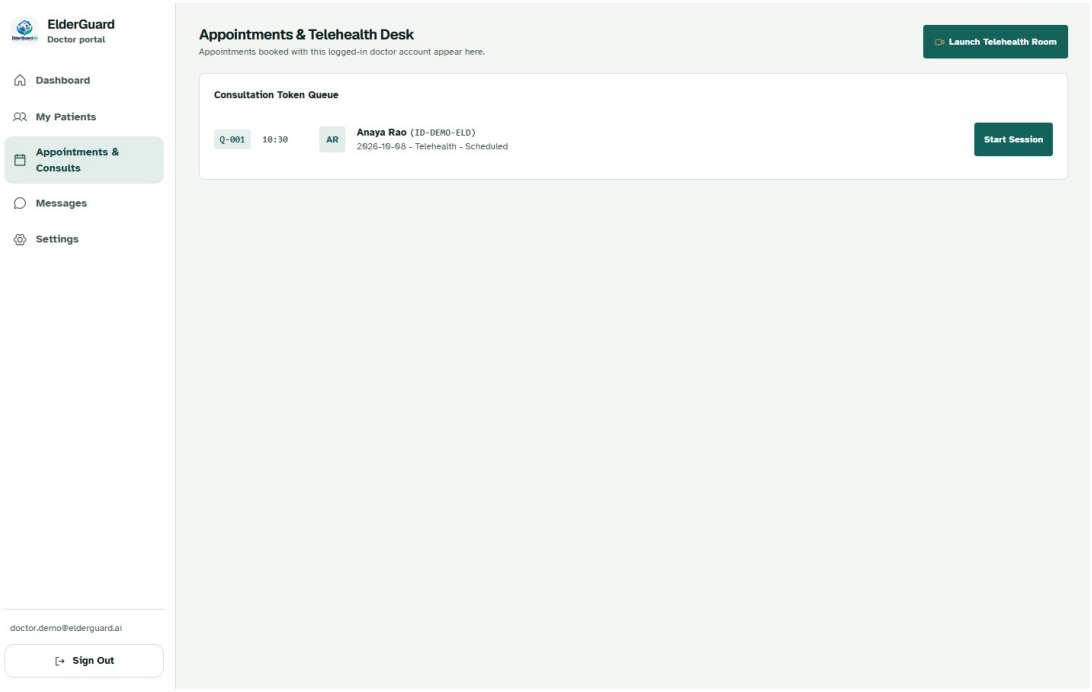


Figure A.10: Appointments and telehealth desk

A.6 Hospital Portal

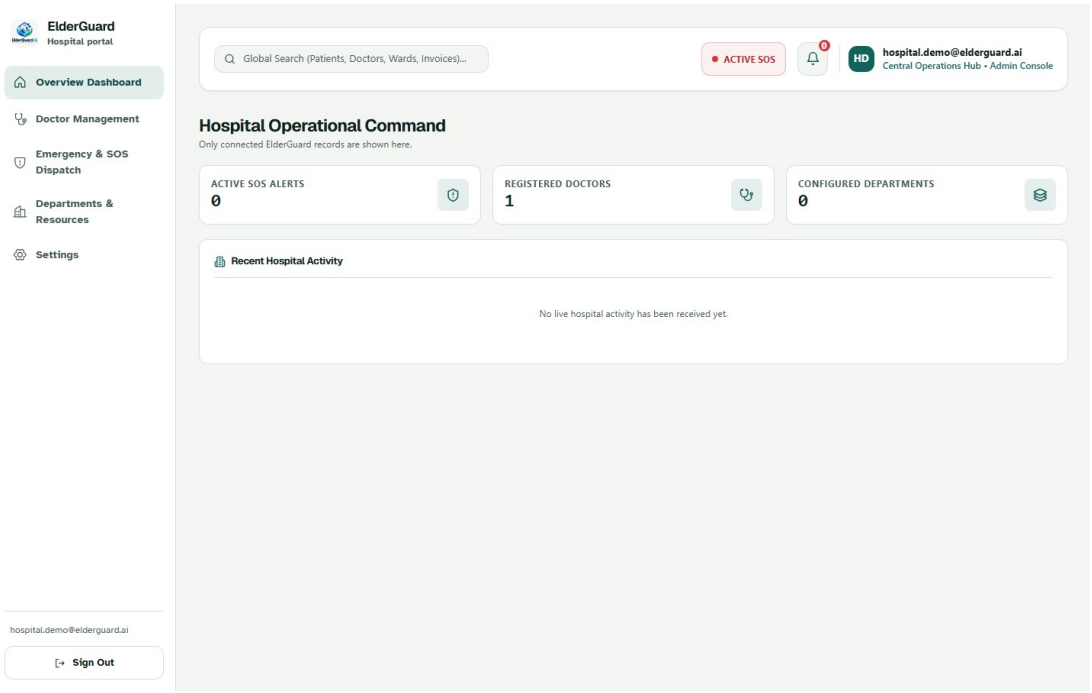


Figure A.11: Hospital operational command

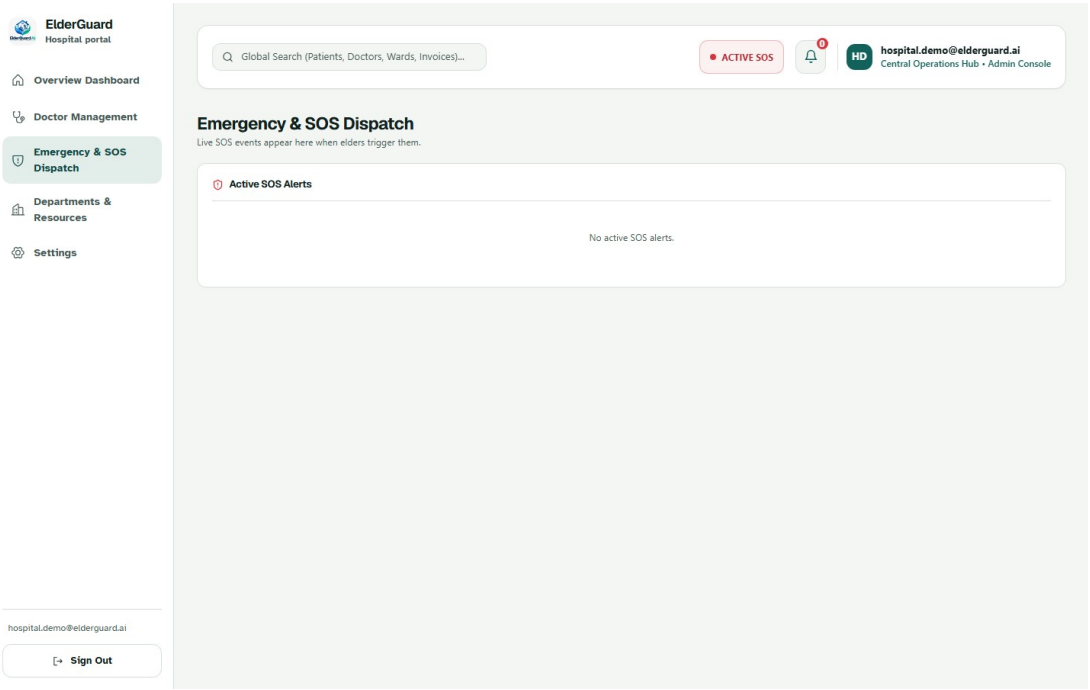


Figure A.12: Emergency and SOS dispatch

A.7 Pharmacy Portal

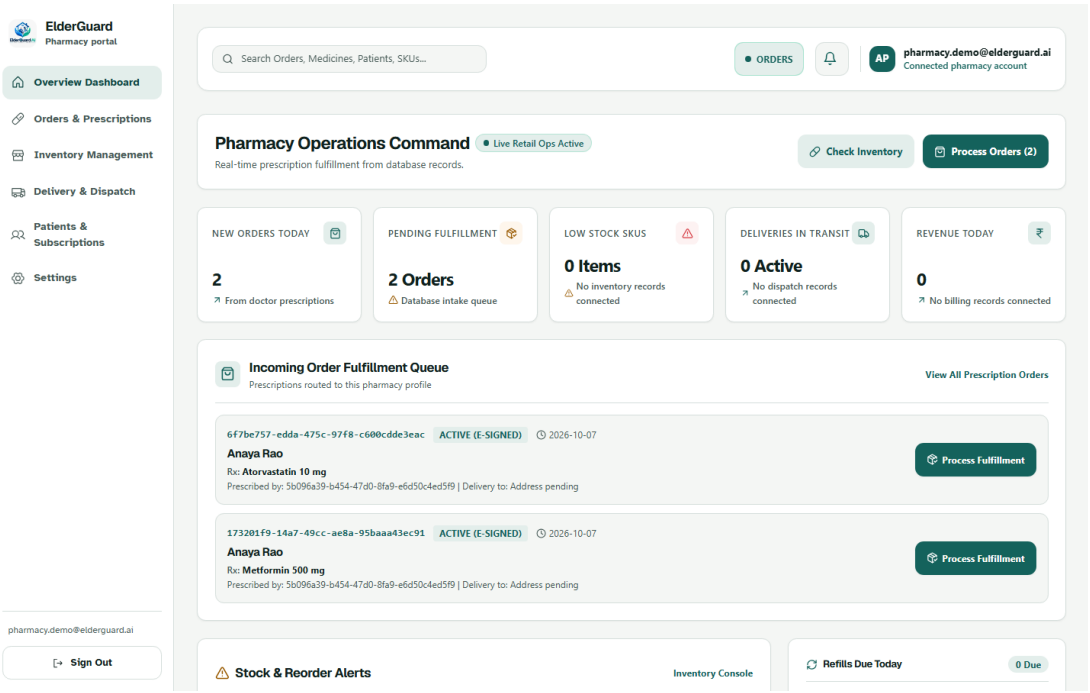


Figure A.13: Pharmacy operations command

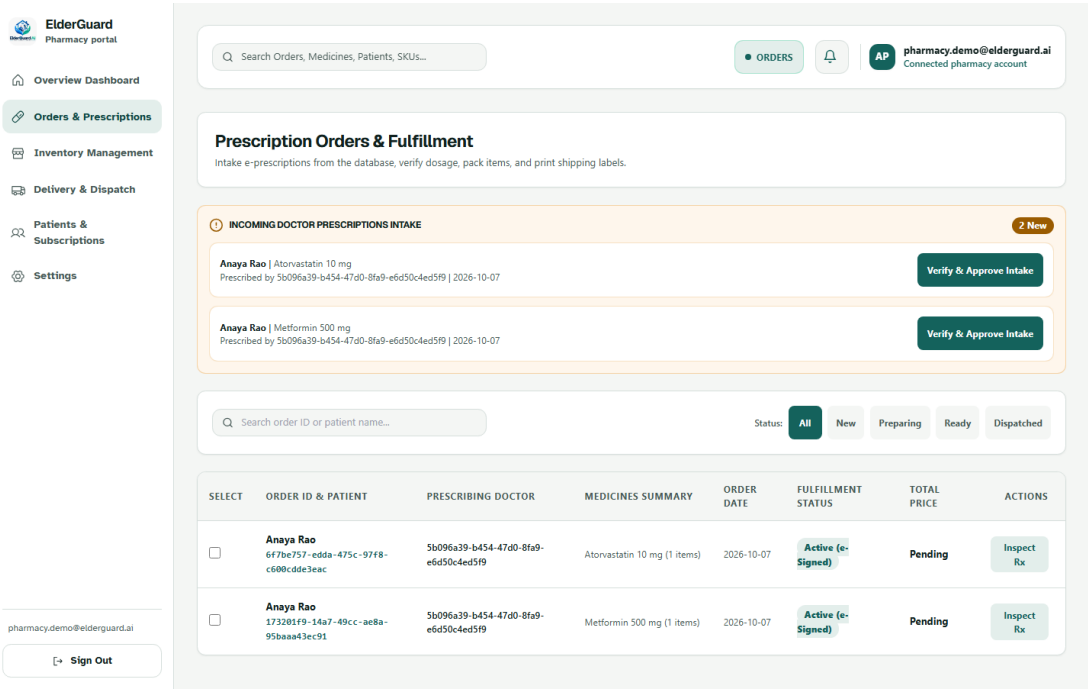


Figure A.14: Prescription orders and fulfilment

Appendix B – Core Functionality Code

B.1 Introduction

Short excerpts of the core logic are shown. The complete source is in the project repository.

B.2 Password hashing (backend/app/core/security.py)

```
def hash_password(password: str) -> str:
    """Hashes a plain text password using bcrypt."""
    salt = bcrypt.gensalt()
    return bcrypt.hashpw(password.encode("utf-8"), salt).decode("utf-8")
```

B.3 SOS escalation sweep schedule (backend/app/emergency/scheduler.py)

```
emergency_scheduler.add_job(
    escalation_sweep, "interval",
    seconds=(5 if SOS_DEMO_SECONDS > 0 else 30),
    id="sos_escalation_sweep")
```

B.4 Medicine reminder sweeps (backend/app/medicines/scheduler.py)

```
scheduler.add_job(sweep_generate_sessions, "interval", seconds=60)
scheduler.add_job(sweep_start_ringing,     "interval", seconds=15)
scheduler.add_job(sweep_check_timeouts,    "interval", seconds=15)
```

B.5 Demo-speed SOS

```
# root package.json
"dev:demo": "set SOS_DEMO_SECONDS=15&& npm run dev"
```

B.6 Code Summary

The excerpts show the separation of concerns: secure credentials, scheduled safety jobs and a configuration switch for demonstration.

Project Link

Source code: https://github.com/Lokesh848-code/Elder_guard

Run locally with the repository's dev script: npm run dev (front end on port 5173, API on port 8000). Demo accounts are created by backend/seed_connected_demo.py.

References

[1] FastAPI Documentation. https://fastapi.tiangolo.com/ – used for the REST and WebSocket back end.

[2] React Documentation. https://react.dev/ – used for the component-based front end.

[3] Vite Documentation. https://vitejs.dev/ – used for the development server and build.

[4] Tailwind CSS Documentation. https://tailwindcss.com/docs – used for design tokens and styling.

[5] Motion (framer-motion) Documentation. https://motion.dev/ – used for interface animation.

[6] SQLAlchemy Documentation. https://docs.sqlalchemy.org/ – used for database access.

[7] APScheduler Documentation. https://apscheduler.readthedocs.io/ – used for scheduled jobs.

[8] Supabase Documentation. https://supabase.com/docs – hosted PostgreSQL option.

[9] Playwright Documentation. https://playwright.dev/ – used for end-to-end testing.

[10] axe-core by Deque Systems. https://github.com/dequelabs/axe-core – used for automated accessibility testing.

[11] W3C, Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/ – accessibility criteria.

[12] Groq API Documentation. https://console.groq.com/docs – hosted language model for the AI companion.

[13] Government of India, Digital Personal Data Protection Act, 2023 – privacy considerations for health data.

[14] Reference reports: “Finora AI” and “MailWorm” mini project reports, SNS College of Technology, 2026 – report structure and format.